

Python 程序设计

目录

前言.....	I
第1章 Python 概述.....	1
1.1 Python 简介.....	1
1.1.1 Python 的发展.....	1
1.1.2 Python 的特点.....	2
1.1.3 Python 的版本.....	3
1.2 Python 开发环境.....	3
1.2.1 Python 语言解释器的下载与安装.....	3
1.2.2 IDLE.....	5
1.2.3 PyCharm.....	6
1.3 编写并运行一个 Python 程序.....	11
1.3.1 交互式执行.....	11
1.3.2 文件式执行.....	12
1.3.3 使用 PyCharm 编写第一个程序.....	13
1.4 安装扩展库的方法.....	15
1.5 习题.....	18
第2章 Python 语言基础.....	19
2.1 常量.....	19
2.2 数据类型.....	19
2.2.1 数值类型.....	19
2.2.2 字符串.....	20
2.2.3 布尔类型.....	21
2.2.4 复合数据类型.....	21
2.3 变量.....	23
2.4 基本运算.....	25
2.4.1 算术运算.....	25
2.4.2 位运算*.....	28
2.4.3 运算优先级.....	30
2.5 常用系统函数.....	30
2.5.1 常用内置函数.....	30
2.5.2 常用库函数.....	31
2.6 Python 的帮助系统.....	35
2.7 习题.....	35
第3章 顺序结构.....	37
3.1 Python 程序的编码规范.....	37
3.1.1 一个 Python 程序.....	37
3.1.2 注释.....	38
3.1.3 缩进.....	38
3.1.4 语句续行与分隔.....	38
3.2 赋值语句.....	39
3.2.1 赋值语句.....	39
3.2.2 变量的共享引用*.....	41
3.2.3 对象的垃圾回收机制*.....	47

3.3 数据的输入/输出.....	48
3.3.1 标准输入/输出.....	48
3.3.2 格式化输出.....	51
3.4 顺序结构程序举例.....	54
3.5 习题.....	56
第4章 选择结构.....	59
4.1 条件表达式.....	59
4.1.1 关系运算.....	59
4.1.2 逻辑运算.....	60
4.1.3 测试运算及函数.....	61
4.2 条件运算符.....	62
4.3 选择结构的实现.....	63
4.3.1 单分支选择结构.....	63
4.3.2 双分支选择结构.....	64
4.3.3 多分支选择结构.....	65
4.3.4 选择结构的嵌套.....	66
4.4 选择结构程序举例.....	67
4.5 习题.....	69
第5章 循环结构.....	错误!未定义书签。
5.1 while 循环结构.....	错误!未定义书签。
5.2 for 循环结构.....	错误!未定义书签。
5.3 循环控制语句.....	错误!未定义书签。
5.3.1 break 语句.....	错误!未定义书签。
5.3.2 continue 语句.....	错误!未定义书签。
5.3.3 pass 语句.....	错误!未定义书签。
5.4 循环的嵌套.....	错误!未定义书签。
5.5 循环结构程序举例.....	错误!未定义书签。
5.6 习题.....	错误!未定义书签。
第6章 列表与元组.....	错误!未定义书签。
6.1 序列.....	错误!未定义书签。
6.2 列表.....	错误!未定义书签。
6.2.1 列表的创建、元素引用及删除.....	错误!未定义书签。
6.2.2 运算符对列表的操作.....	错误!未定义书签。
6.2.3 内置函数对列表的操作.....	错误!未定义书签。
6.2.4 列表对象的常用方法.....	错误!未定义书签。
6.2.5 列表的切片.....	错误!未定义书签。
6.2.6 列表推导式.....	错误!未定义书签。
6.3 元组.....	错误!未定义书签。
6.3.1 元组的创建、元素引用与删除.....	错误!未定义书签。
6.3.2 元组的常用操作.....	错误!未定义书签。
6.3.3 生成器推导式.....	错误!未定义书签。
6.3.4 元组与列表的异同点.....	错误!未定义书签。
6.4 应用举例.....	错误!未定义书签。
6.5 习题.....	错误!未定义书签。

第 7 章 字典与集合	错误!未定义书签。
7.1 字典	错误!未定义书签。
7.1.1 字典概述	错误!未定义书签。
7.1.2 字典的创建、元素引用及删除	错误!未定义书签。
7.1.3 运算符对字典的操作	错误!未定义书签。
7.1.4 内置函数对字典的操作	错误!未定义书签。
7.1.5 字典对象的常用方法	错误!未定义书签。
7.1.6 字典推导式	错误!未定义书签。
7.1.7 字典的遍历	错误!未定义书签。
7.2 集合	错误!未定义书签。
7.2.1 集合的创建与删除	错误!未定义书签。
7.2.2 集合的运算	错误!未定义书签。
7.2.3 内置函数对集合的操作	错误!未定义书签。
7.2.4 集合对象的常用方法	错误!未定义书签。
7.2.5 集合的遍历和推导式	错误!未定义书签。
7.3 应用举例	错误!未定义书签。
7.4 习题	错误!未定义书签。
第 8 章 字符串与正则表达式	错误!未定义书签。
8.1 字符串	错误!未定义书签。
8.1.1 字符串的创建、索引及删除	错误!未定义书签。
8.1.2 字符串编码	错误!未定义书签。
8.1.3 运算符对字符串的操作	错误!未定义书签。
8.1.4 内置函数对字符串的操作	错误!未定义书签。
8.1.5 字符串的遍历与切片	错误!未定义书签。
8.1.6 字符串对象的常用方法	错误!未定义书签。
8.1.7 字符串常量	错误!未定义书签。
8.2 正则表达式	错误!未定义书签。
8.2.1 正则表达式语言概述	错误!未定义书签。
8.2.2 正则表达式元字符	错误!未定义书签。
8.2.3 常用的正则表达式	错误!未定义书签。
8.2.4 正则表达式模块	错误!未定义书签。
8.3 应用举例	错误!未定义书签。
8.4 习题	错误!未定义书签。
第 9 章 函数与模块	错误!未定义书签。
9.1 函数的定义与调用	错误!未定义书签。
9.1.1 函数的定义	错误!未定义书签。
9.1.2 函数的调用	错误!未定义书签。
9.1.3 lambda 表达式和匿名函数	错误!未定义书签。
9.2 函数的参数传递	错误!未定义书签。
9.2.1 参数的传递方式	错误!未定义书签。
9.2.2 参数的类型	错误!未定义书签。
9.3 函数的返回值	错误!未定义书签。
9.3.1 return 语句和函数返回值	错误!未定义书签。
9.3.2 多条 return 语句	错误!未定义书签。

9.3.3 返回多个值.....	错误!未定义书签。
9.4 变量的作用域.....	错误!未定义书签。
9.4.1 局部变量.....	错误!未定义书签。
9.4.2 全局变量.....	错误!未定义书签。
9.5 递归函数.....	错误!未定义书签。
9.6 内嵌函数*.....	错误!未定义书签。
9.7 装饰器*.....	错误!未定义书签。
9.7.1 无参数装饰器.....	错误!未定义书签。
9.7.2 有参数装饰器.....	错误!未定义书签。
9.7.3 多重装饰器.....	错误!未定义书签。
9.8 模块.....	错误!未定义书签。
9.8.1 什么是模块.....	错误!未定义书签。
9.8.2 Python 的程序架构.....	错误!未定义书签。
9.8.3 模块搜索路径*.....	错误!未定义书签。
9.8.4 模块的有条件执行.....	错误!未定义书签。
9.9 习题.....	错误!未定义书签。
第 10 章 文件及目录操作.....	错误!未定义书签。
10.1 文件概述.....	错误!未定义书签。
10.2 文件的打开与关闭.....	错误!未定义书签。
10.2.1 打开文件.....	错误!未定义书签。
10.2.2 关闭文件.....	错误!未定义书签。
10.3 文本文件的读写.....	错误!未定义书签。
10.3.1 读取文本文件.....	错误!未定义书签。
10.3.2 文本文件的写入.....	错误!未定义书签。
10.4 CSV 文件的读写.....	错误!未定义书签。
10.4.1 CSV 文件简介.....	错误!未定义书签。
10.4.2 读取 CSV 文件.....	错误!未定义书签。
10.4.3 写入 CSV 文件.....	错误!未定义书签。
10.5 文件内的移动.....	错误!未定义书签。
10.6 二进制文件的读写*.....	错误!未定义书签。
10.6.1 pickle 模块.....	错误!未定义书签。
10.6.2 struct 模块.....	错误!未定义书签。
10.7 文件与目录管理*.....	错误!未定义书签。
10.7.1 文件管理.....	错误!未定义书签。
10.7.2 目录管理.....	错误!未定义书签。
10.8 文件压缩*.....	错误!未定义书签。
10.8.1 文件压缩.....	错误!未定义书签。
10.8.2 解压文件.....	错误!未定义书签。
10.9 文件操作应用举例.....	错误!未定义书签。
10.10 习题.....	错误!未定义书签。
第 11 章 面向对象程序设计.....	错误!未定义书签。
11.1 面向对象程序设计基础.....	错误!未定义书签。
11.1.1 面向过程与面向对象.....	错误!未定义书签。
11.1.2 面向对象的基本概念.....	错误!未定义书签。

11.2 类与对象.....	错误!未定义书签。
11.2.1 类的定义.....	错误!未定义书签。
11.2.2 对象的创建和使用.....	错误!未定义书签。
11.3 属性.....	错误!未定义书签。
11.3.1 实例属性和类属性.....	错误!未定义书签。
11.3.2 私有属性和公有属性.....	错误!未定义书签。
11.3.3 特殊属性与方法.....	错误!未定义书签。
11.4 方法.....	错误!未定义书签。
11.4.1 实例方法.....	错误!未定义书签。
11.4.2 类方法.....	错误!未定义书签。
11.4.3 静态方法.....	错误!未定义书签。
11.5 继承和多态.....	错误!未定义书签。
11.5.1 继承.....	错误!未定义书签。
11.5.2 多态.....	错误!未定义书签。
11.6 重载*.....	错误!未定义书签。
11.7 面向对象程序设计应用举例.....	错误!未定义书签。
11.8 习题.....	错误!未定义书签。
第 12 章 错误和异常处理.....	错误!未定义书签。
12.1 程序的错误.....	错误!未定义书签。
12.2 异常处理.....	错误!未定义书签。
12.2.1 异常概述.....	错误!未定义书签。
12.2.2 try-except 语句.....	错误!未定义书签。
12.2.3 自定义异常类*.....	错误!未定义书签。
12.3 断言处理.....	错误!未定义书签。
12.3.1 断言处理概述.....	错误!未定义书签。
12.3.2 assert 语句和 AssertionError 类.....	错误!未定义书签。
12.3.3 启用/禁用断言*.....	错误!未定义书签。
12.4 Python 的调试工具 pdb*.....	错误!未定义书签。
12.5 习题.....	错误!未定义书签。
第 13 章 图形界面程序设计*.....	错误!未定义书签。
13.1 图形用户界面概述.....	错误!未定义书签。
13.1.1 图形用户界面.....	错误!未定义书签。
13.1.2 tkinter 库创建图形用户界面.....	错误!未定义书签。
13.2 常用控件.....	错误!未定义书签。
13.2.1 概述.....	错误!未定义书签。
13.2.2 按钮.....	错误!未定义书签。
13.2.3 标签和消息.....	错误!未定义书签。
13.2.4 文本框.....	错误!未定义书签。
13.2.5 单选按钮和复选框.....	错误!未定义书签。
13.2.6 列表框.....	错误!未定义书签。
13.2.7 滚动条和刻度条.....	错误!未定义书签。
13.2.8 框架.....	错误!未定义书签。
13.2.9 菜单.....	错误!未定义书签。
13.2.10 ttk 子模块.....	错误!未定义书签。

13.3 对话框.....	错误!未定义书签。
13.3.1 弹出消息框.....	错误!未定义书签。
13.3.2 文件选择对话框.....	错误!未定义书签。
13.3.3 颜色选择对话框.....	错误!未定义书签。
13.3.4 自定义对话框.....	错误!未定义书签。
13.4 布局管理器.....	错误!未定义书签。
13.4.1 tkinter 模块绘图坐标系	错误!未定义书签。
13.4.2 pack 布局管理器	错误!未定义书签。
13.4.3 grid 布局管理器	错误!未定义书签。
13.4.4 place 布局管理器	错误!未定义书签。
13.5 事件处理.....	错误!未定义书签。
13.5.1 tkinter 的事件处理机制	错误!未定义书签。
13.5.2 event 类.....	错误!未定义书签。
13.5.3 事件描述.....	错误!未定义书签。
13.5.4 用 bind 方法进行事件绑定.....	错误!未定义书签。
13.6 图形用户界面应用举例.....	错误!未定义书签。
13.7 习题.....	错误!未定义书签。
第 14 章 数据库应用开发*	错误!未定义书签。
14.1 数据库技术基础.....	错误!未定义书签。
14.1.1 数据库基本概念.....	错误!未定义书签。
14.1.2 关系数据库	错误!未定义书签。
14.1.3 数据库应用系统的开发步骤.....	错误!未定义书签。
14.1.4 数据库设计	错误!未定义书签。
14.2 MySQL 数据库	错误!未定义书签。
14.2.1 安装 MySQL	错误!未定义书签。
14.2.2 创建数据库	错误!未定义书签。
14.2.3 MySQL 的数据类型	错误!未定义书签。
14.2.4 创建表.....	错误!未定义书签。
14.2.5 插入数据.....	错误!未定义书签。
14.2.6 查看表.....	错误!未定义书签。
14.2.7 修改数据.....	错误!未定义书签。
14.2.8 删除数据.....	错误!未定义书签。
14.2.9 删除表.....	错误!未定义书签。
14.3 Python 操作 MySQL 数据库举例.....	错误!未定义书签。
14.3.1 PyMySQL 模块.....	错误!未定义书签。
14.3.2 数据库应用开发举例.....	错误!未定义书签。
14.3.3 数据库应用开发步骤详解.....	错误!未定义书签。
14.3.4 数据库表的插入、更新和删除操作.....	错误!未定义书签。
14.4 习题	错误!未定义书签。
第 15 章 Python 网站开发*	错误!未定义书签。
15.1 Web 开发框架概述.....	错误!未定义书签。
15.1.1 常用的 Web 开发框架.....	错误!未定义书签。
15.1.2 MVC 和 MTV 模式简介.....	错误!未定义书签。
15.2 Django 开发环境的搭建.....	错误!未定义书签。

15.2.1 Django 框架的安装.....	错误!未定义书签。
15.2.2 创建第一个 Django 项目	错误!未定义书签。
15.3 Django 框架的应用	错误!未定义书签。
15.3.1 数据库配置	错误!未定义书签。
15.3.2 创建数据模型.....	错误!未定义书签。
15.3.3 创建视图.....	错误!未定义书签。
15.3.4 模板系统.....	错误!未定义书签。
15.3.5 部署 Django 项目	错误!未定义书签。
15.4 Django 框架的高级应用.....	错误!未定义书签。
15.4.1 Session.....	错误!未定义书签。
15.4.2 国际化.....	错误!未定义书签。
15.5 Web 开发应用案例——小型图书管理系统.....	错误!未定义书签。
15.6 习题	错误!未定义书签。

第1章 Python 概述

1.1 Python 简介

1.1.1 Python 的发展

1. 程序设计概述

目前，计算机已经深入到社会的各个领域，成为人们日常工作、生活、学习的必备工具。计算机是一种具有存储程序、执行程序能力的电子设备，计算机所有能力都是通过执行程序来实现的。程序就是人们把需要做的工作写成一定形式的指令序列，并把它存储在计算机内部存储器中，当人们给出完成工作的命令后，计算机就按预设的指令序列自动进行相应操作，从而完成相应的工作。人们把这种可以连续执行的一条条指令的序列称为“程序”。编写程序的过程就称为“程序设计”。为了使计算机能够正确识别和执行程序，指令是不能随意编写的，必须有一定的规则。这些规则包含了一系列的文法和语法的要求，按照这些规则编写的程序才能够被计算机理解执行，所以它是人和计算机之间的交流语言。这种语言类似于人与人之间交流的语言，虽然没有人类语言那么复杂，但逻辑上要求更加严格，符合这些规则的语言被称之为“程序设计语言”。

计算机可以直接识别和运行的语言称为“机器语言”，这是一种二进制代码语言。每台机器的指令，其格式和代码所代表的含义都是有硬性规定的，比如某种计算机的指令 `10110110 00000000` 表示让计算机进行一次加法操作；而指令 `10110101 00000000` 则表示进行一次减法操作。它们的前八位表示操作码，后八位则表示地址码。由于硬件设计不同，机器语言对不同型号的计算机来说一般是不同的。用机器语言编程，就是从 CPU 的指令系统中挑选合适的指令组成一个指令系列的过程。

由于机器语言与人们日常生活中使用的语言差距过大，而且大量的规则都和具体的计算机硬件设计和实现相关，所以使用机器语言编写程序的难度很大。为了降低编写程序的难度，人们发明了一些更加接近人类日常语言的程序设计语言。根据与人类语言的接近程度不同，可把这些语言分为高级语言、中级语言和低级语言。低级语言最接近机器语言，学习和使用难度都比较大；高级语言最接近人类语言，学习和使用难度相对低级语言来说要容易得多，应用也最为广泛。目前常见的高级语言有 C、Java、C++、C#、Python、PHP 等，并且还在不断涌现新的程序设计语言。

由某种程序设计语言编写的程序称为“源程序”，高级语言源程序不能被计算机识别和执行，必须翻译成机器语言指令才能被计算机执行，其翻译过程如图 1-1 所示。

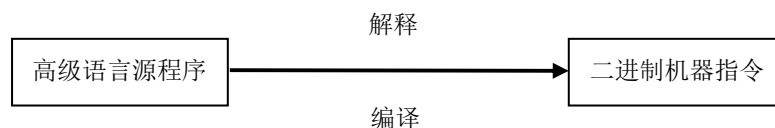


图 1-1 程序的翻译过程示意图

2. Python 的起源

Python 的创始人是荷兰人 Guido von Rossum。过去因为硬件能力的限制，程序的设计原则是让机器更快地运行。为了增进效率，程序员必须像计算机一样思考，以便能写出更符合机器口味的程序。这种编程方式让 Guido 感到苦恼，虽然 Guido 知道如何用 C 语言实现一个

功能，但整个编写过程需要耗费大量的时间。

Guido 的另一个编程选择是 UNIX 系统的解释器（interpreter）Bourne Shell。UNIX 的管理员们常常用 Shell 写一些简单的脚本，以进行一些系统的维护工作，比如定期备份、文件系统管理等。Shell 可以像胶水一样，将 UNIX 下的许多功能连接在一起。C 语言下许多上百行的程序，在 Shell 下只用几行就可以完成。然而，Shell 的本质是调用命令。它不是一个真正的语言。比如说，Shell 没有数值型的数据类型，要实现加法运算都很复杂。

Guido 希望有一种语言既能够像 C 语言那样，可以全面调用计算机的功能接口，又可以像 Shell 那样轻松地编程。ABC 语言让 Guido 看到了希望。ABC 语言以教学为目的，与当时大部分的语言不同，它的目标是“让用户感觉更好”。ABC 语言希望让语言变得容易阅读、容易使用、容易记忆、容易学习，并以此来激发人们学习编程的兴趣。

然而 ABC 语言最终没有流行起来。因为 ABC 语言编译器需要在比较高配置的电脑上才能运行，而这些电脑的使用者通常都精通计算机，他们更多考虑的是程序的效率，而非它的学习难度。除了硬件上的困难外，ABC 语言的设计也存在一些致命的问题，比如，不能直接进行 IO 操作、可拓展性差、过度革新等。

1989 年，为了打发圣诞节假期，Guido 开始写 Python 语言的编译/解释器。他希望这个叫做 Python 的语言能实现他的理念——设计出一种 C 和 Shell 之间、功能全面、易学易用、可拓展的语言。

3. Python 的诞生

1991 年，第一个 Python 编译器/解释器诞生。它是用 C 语言实现的，并能够调用 C 库（.so 文件）。从一出生，Python 已经具有类（class）、函数（function）、异常（exception），以及包括列表（list）和词典（dictionary）在内的核心数据类型和模块（module）等语法元素，并能以此为基础进行拓展。

Python 从一开始就特别在意可拓展性（extensibility）。Python 可以在多个层次上拓展。从高层可以导入.py 文件。在底层可以引用 C 语言的库。Python 程序员可以快速地用 Python 写.py 文件作为拓展模块。当性能是考虑的重要因素时，Python 程序员又可以深入底层，写 C 程序并编译为.so 文件导入到 Python 中使用。Python 就好像使用钢构建房一样，先规定好大的框架，程序员可以在此框架下自由地拓展或更改。

Python 的诞生可谓生逢其时。从上世纪 90 年代初，个人计算机开始普及，计算机的性能也有了很大的提高，硬件厂商甚至渴望高需求软件的出现，以带动硬件的更新换代。程序员开始关注计算机的易用性，图形化界面的备受推崇就是个佐证。我们还记得，ABC 失败的一个重要原因是硬件的性能限制。然而现在硬件性能已经不再是个瓶颈，语言的易用性被提到了一个新的高度，所以许多人开始转向易于使用的 Python。

1.1.2 Python 的特点

- （1）简单。Python 语言的语法简单，易读、易用、易记、易习。阅读一个良好的 Python 程序就好像是在读英语一样，使得 Python 成为一门非常容易入门的语言。Python 简洁的语法格式，可以让程序员专注于解决问题而不是纠缠语法细节。
- （2）开源性。Python 是 FLOSS（自由/开放源码软件）之一。简单地说，你可以自由地发布这个软件的拷贝，阅读它的源代码，对它做改动，把它的一部分用于新的自由软件中。Python 希望看到更加优秀的人创造并经常改进它。
- （3）高级语言。当用 Python 语言编写程序的时候，你无需考虑诸如如何管理程序使用的内存一类的底层细节。
- （4）可移植性。由于它的开源本质，Python 已经被移植在许多平台上。如果你小心地避免使用依赖于系统的特性，那么你的 Python 程序无需修改就可以在任何平

台上运行。

- (5) 解释执行。Python 解释器把源代码转换成字节码的中间形式，然后再把它翻译成计算机使用的机器语言并运行。你不需要操心如何编译程序，如何确保链接装载正确的库等，所有这一切在 Python 中都非常简单。
- (6) 面向对象编程。Python 既支持面向过程的函数编程也支持面向对象的抽象编程。在面向过程的语言中，程序是由过程或者可重用代码的函数构建起来的。在面向对象的语言中，程序是由数据和功能组合在一起的对象构建起来的。与其他主要的语言如 C++ 和 Java 相比，Python 以一种非常强大而又简单的方式实现面向对象编程。
- (7) 可扩展性。如果你需要一段关键代码运行得更快或者希望某些算法不公开，你可以把你的部分程序用 C 或 C++ 编写，然后在你的 Python 程序中引用它们。
- (8) 可嵌入性。可以把 Python 嵌入到 C 或 C++ 程序中，向程序用户提供脚本功能。
- (9) 丰富的库。Python 标准库已经很庞大，同时，Python 还有可定义的第三方库可以使用。这些库可以帮助你处理各种工作，包括正则表达式、文档生成、单元测试、线程、数据库、网页浏览器、CGI、FTP、电子邮件、XML、XML-RPC、HTML、WAV 文件、密码系统、GUI（图形用户界面）、Tk 和其他与系统有关的操作。

1.1.3 Python 的版本

Python 2 发布于 2000 年年底，较之先前版本，这是一种更加清晰和更具包容性的语言。先前版本的 Python 应用的是 PEP（Python 增强协议）。Python 2 包括了更多的程序性功能，比如能自动化地管理内存的循环检测垃圾收集器，增加了对 Unicode 的支持以实现字符的标准化，并采用列表综合的方式，可以在现有列表基础上创建新列表。随着 Python 2 的不断发展，更多的功能被添加进来，包括将 Python 的类型和类在 Python 2.2 版本中统一为一层。

作为一项重大改革，Python 3 于 2008 年年末发布，以解决和修正以前语言版本的内在设计缺陷。Python 3 开发的重点是清理代码库并删除冗余，清晰地表明只能用一种方式来执行给定的任务。

起初，Python 3 的采用很缓慢，因为它不能向后兼容 Python 2，这就迫使人们需要决定该使用哪个版本的语言。此外，许多封装库只适用于 Python 2，但是由于 Python 的开发团队重申了终止对 Python 2 的支持，促使更多的库被移植到 Python 3 上来。

在 2008 年 Python 3.0 发布之后，Python 2.7 于 2010 年 7 月 3 日发布，并计划作为 2.x 版本的最后一版。发布 Python 2.7 的目的在于，通过提供一些测量两者之间兼容性的措施，使 Python 2.x 的用户更容易将功能移植到 Python 3 上。现在所说的 Python 2，通常是指 Python 2.7 版本，它是 Python 2 最常用的版本。而对 Python 2 的后续开发，包括现在最主要的 bug 修复，将在 2020 年完全停止。鉴于这个原因，本文使用的版本是 Python 3。

1.2 Python 开发环境

集成开发环境（IDE，Integrated Development Environment）是用于提供程序开发环境的应用程序，包括代码编辑器、编译/解释器、调试器和图形用户界面等工具。IDE 集成了代码编写功能、分析功能、编译/解释功能、调试功能等，具备这些功能的软件或者软件套（组）都可以叫做集成开发环境。

1.2.1 Python 语言解释器的下载与安装

解释器（Interpreter）是一种能够把一行行高级语言源码直译成机器码后让 CPU 执行的程序。当我们写完 Python 代码后，需要 Python 解释器去转换并执行。Python 语言主网上提

供了不同版本的解释器软件，用户根据需要下载即可。

本书以 Python 3.5.2 版本为例介绍 Python 解释器的安装。在浏览器中输入 Python 官网下载页面的地址 <https://www.python.org/downloads/>，找到页面中部的版本列表（如图 1-2），单击条目“Python 3.5.2...”的“Download”，出现 Python 3.5.2 版本的相关信息，在页面中找到如图 1-3 的下载文件清单，单击“Windows x86-64 executable installer”下载安装 Python3.5.2 的安装文件。

下载完后双击安装文件 Python-3.5.2-amd64.exe，注意务必勾选“Add Python 3.5 to PATH”复选框，单击“Install Now”开始安装，如图 1-4 所示。安装成功后将显示如图 1-5 所示的界面。

Python 安装包会在系统中安装一批与 Python 开发和运行相关的文件，其中最重要的两个程序是 Python Shell 和 Python 集成开发环境 IDLE，安装完成后，就可以在 Windows 的开始菜单中看到这两个 Python 程序的图标。

Looking for a specific release?

Python releases by version number:

Release version	Release date		Click for more
Python 3.6.0	2016-12-23	Download	Release Notes
Python 2.7.13	2016-12-17	Download	Release Notes
Python 3.4.5	2016-06-27	Download	Release Notes
Python 3.5.2	2016-06-27	Download	Release Notes
Python 2.7.12	2016-06-25	Download	Release Notes
Python 3.4.4	2015-12-21	Download	Release Notes
Python 3.5.1	2015-12-07	Download	Release Notes
Python 2.7.11	2015-12-05	Download	Release Notes

[View older releases](#)

图 1-2 Python 的下载页面

Files					
Version	Operating System	Description	MD5 Sum	File Size	GPG
Gzipped source tarball	Source release		3fe8434643a78630c61c6464fe2e7e72	20566643	SIG
XZ compressed source tarball	Source release		8906efbacfdc7c3c9198aefafdf159e	15222676	SIG
Mac OS X 32-bit i386/PPC installer	Mac OS X	for Mac OS X 10.5 and later	5ae81eea42bb6758b6d775ebcaf32eda	26250336	SIG
Mac OS X 64-bit/32-bit installer	Mac OS X	for Mac OS X 10.6 and later	11a9f4fc3f6b93e3ffb26c383822a272	24566858	SIG
Windows help file	Windows		24b95be314f7bad1cc5361ae449adc3d	7777812	SIG
Windows x86-64 embeddable zip file	Windows	for AMD64/EM64T/x64	f1c24bb78bd6dd792a73d5ebfbd3b20e	6862200	SIG
Windows x86-64 executable installer	Windows	for AMD64/EM64T/x64	4da6dbc8e43e2249a0892d257e977291	30177896	SIG
Windows x86-64 web-based installer	Windows	for AMD64/EM64T/x64	c35b6526761a9cde4b6dccb4a3d7c60	970224	SIG
Windows x86 embeddable zip file	Windows		ad637a1db7cf91e344318d55c94ad3ca	6048722	SIG
Windows x86 executable installer	Windows		2dd4f428fd8b9c063ba05b5a0c8636c37	29269656	SIG
Windows x86 web-based installer	Windows		aed3ac79b8e2458b84135ecfdca66764	944304	SIG

图 1-3 Python 3.5.2 的下载文件清单

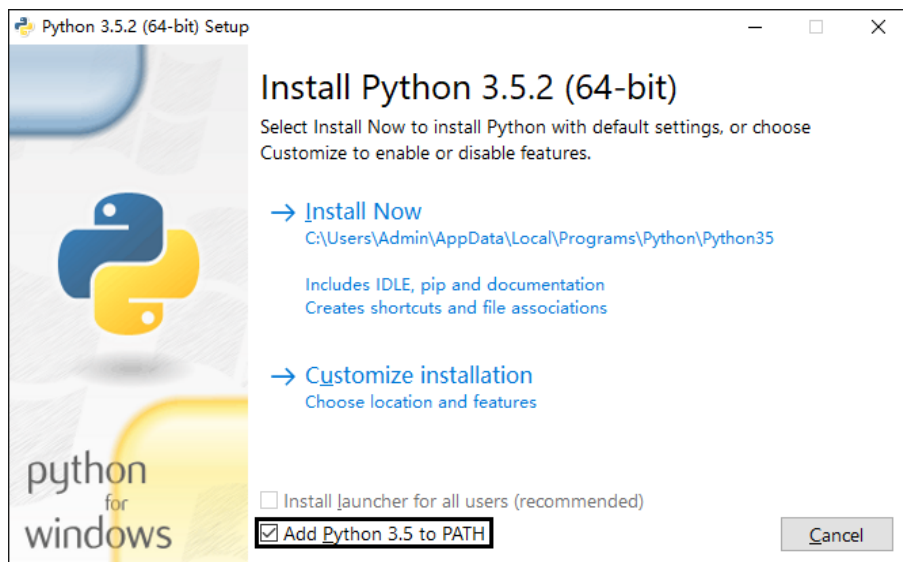


图 1-4 安装程序的启动界面

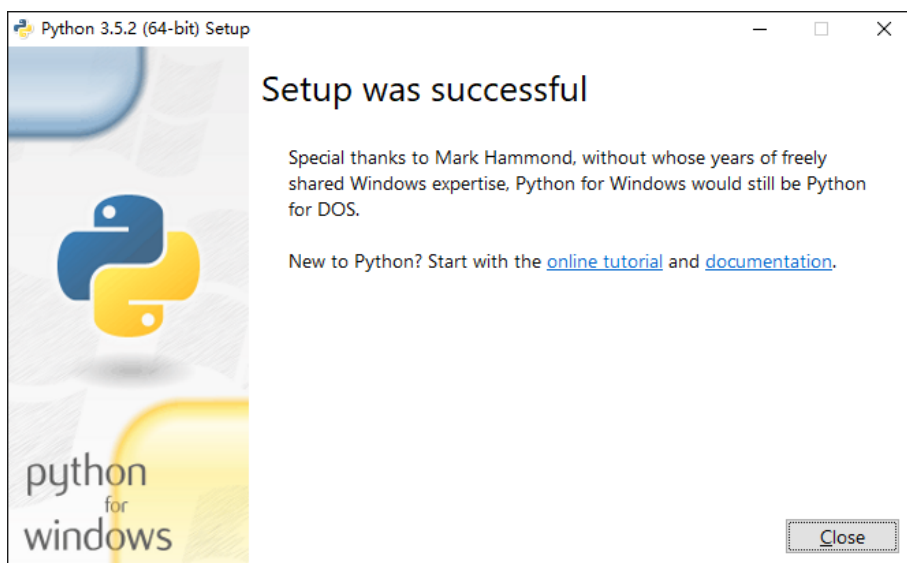


图 1-5 安装程序的安装成功界面

1.2.2 IDLE

IDLE (Python's Integrated DeveLopment Environment, Python 集成开发环境) 是 Python 自带的一个 IDE, 这是一个简洁实用的编辑器, 初学者可以利用它方便地创建、运行和测试 Python 程序。

IDLE 具有许多有用的特性: 如自动缩进、语法高亮显示、标识符快速输入以及程序文件状态提示等。

- 自动缩进

根据语法规则自动缩进。例如: 输入 `if` 关键字所在行末的冒号且按回车键之后, IDLE 会对新一行的输入光标自动缩进。一般情况下, IDLE 将代码缩进一级, 即 4 个空格。如果想改变这个默认的缩进量的话, 可以从“Format”菜单选择“New indent width”项来进行修改。

- 语法高亮显示

IDLE 在用户输入代码时，会自动对代码的不同元素使用不同的颜色进行显示。默认情况下，关键字显示为桔红色，注释显示为红色，字符串为绿色，定义和解释器的输出显示为蓝色，控制台输出显示为棕色。语法高亮显示的好处是，可以让用户更容易区分不同的语法元素，提高程序的可读性；与此同时，语法高亮显示还能在一定程度上降低出错的可能。例如：如果输入的变量名显示为桔红色，则提醒用户该名称与预留的关键字冲突，应该给变量更换个名字。

- 标识符快速输入

当用户输入已出现在当前程序中的标识符单词的一部分后，从“Edit”菜单选择“Expand Word”项，或者直接按【Alt+/】组合键可以自动完成该单词的输入。例如，已有语句：`MynametestNewYear_008 = 100`，即程序使用一个长名字的变量，此后，用户输入 `My` 后按【Alt+/】组合键后，`MynametestNewYear_008` 就会全部出现。

- 程序文件状态提示

新建一个程序文件，IDLE 的标题栏里会显示“Untitled”，提示用户当前文件没有保存。如果正在修改某个原有文件，则标题栏里显示的是当前文件的文件名，若文件中存在新修改且尚未存盘的内容，标题栏的文件名前后会有*星号出现。

1.2.3 PyCharm

PyCharm 是由 JetBrains 打造的一个 Python IDE，带有一整套可以帮助用户在使用 Python 语言开发时提高效率的工具，比如调试、语法高亮、Project 管理、代码跳转、智能提示、自动完成、单元测试、版本控制等。此外，该 IDE 提供了一些高级功能，用于支持 Django 框架下的专业 Web 开发。

1. PyCharm 的安装

撰写本书时，PyCharm 官网上的最新版本是 `pycharm-professional-2018.3.3`，这里介绍的就是该版本的下载及安装过程。官网的页面随时可能会变化，读者在阅读本书时，实际网页的内容可能会不同。

在浏览器中输入 PyCharm 官网地址 <https://www.jetbrains.com/pycharm/>（图 1-6），点击“Download”按钮进入下载页面。PyCharm 有 Professional（专业版）和 Community（社区版）两个版本（如图 1-7），其适用范围正如页面上的简介所示，读者可以根据开发需要选择下载不同的版本。本书使用的是 Professional 版。

双击已下载好的文件 `pycharm-professional-2018.3.3.exe`，按提示信息安装，安装过程见图 1-8 到图 1-13。



图 1-6 PyCharm 主页

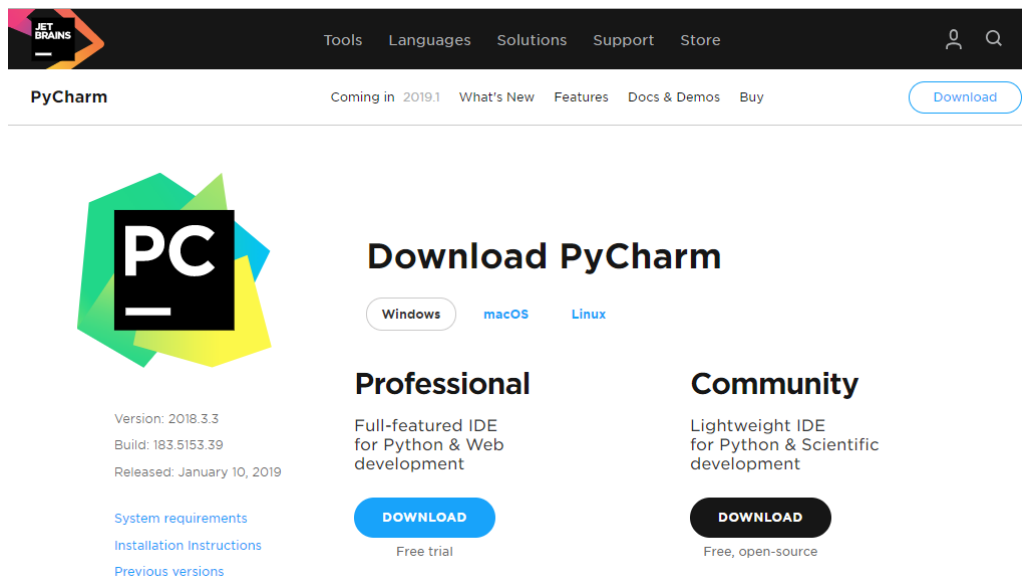


图 1-7 PyCharm 下载页面

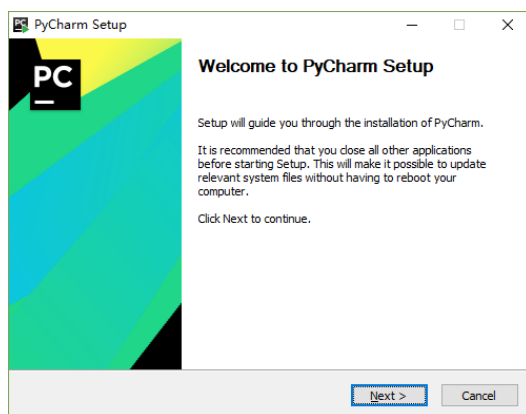


图 1-8 PyCharm 安装向导（一）

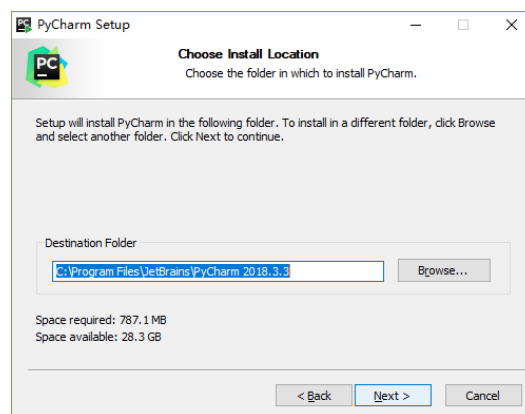


图 1-9 PyCharm 安装向导（二）

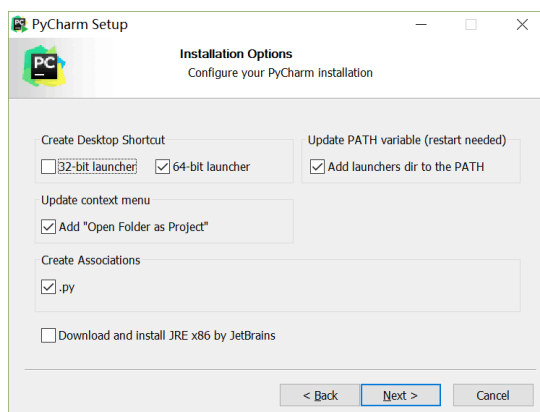


图 1-10 PyCharm 安装向导（三）

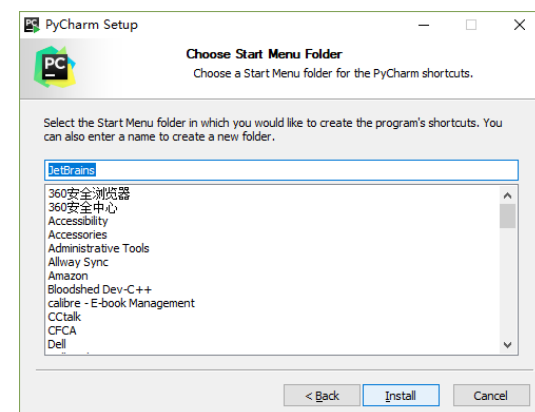


图 1-11 PyCharm 安装向导（四）

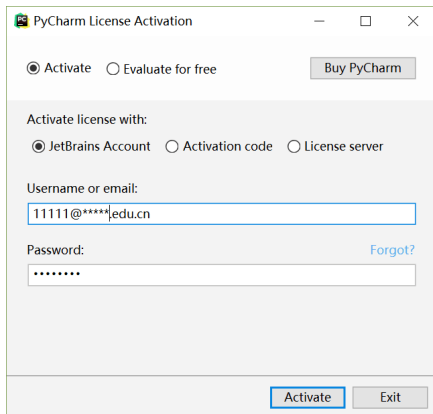


图 1-12 PyCharm 安装向导（五）

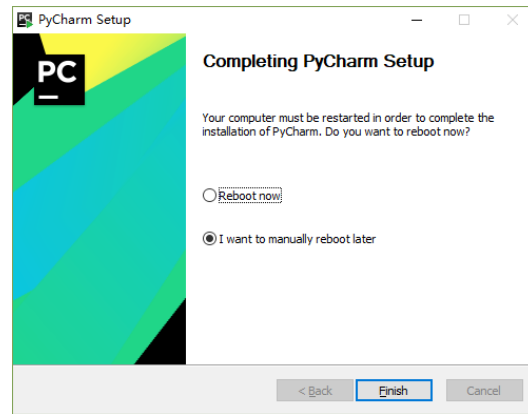


图 1-13 PyCharm 安装向导（六）

2. PyCharm 的激活

对于学生或教师使用者，PyCharm 可以给予免费的授权，但不是永久性的，需要学生提供校园邮箱或是国际学生证（ISIC）。打开网页 <https://www.jetbrains.com/zh/student/#faq-cn>，如下图 1-14，按网页提示操作，可以获得激活邮件，并注册授权账户。



图 1-14 学生免费授权页面

3. PyCharm 的配置

（1）首次启动后的设置

安装完 PyCharm 首次运行时，需要选择软件的界面风格（如图 1-15）。配置选择后，就会启动 PyCharm，出现如图 1-16 的启动欢迎界面，以后每次启动 PyCharm 都会出现此界面。用户可以根据需要选择打开已有的文件/项目或新建文件/项目。

（2）配置菜单和工具栏

在图 1-16 启动界面的右下角有一个“Configure”下列菜单，点击“Settings”命令项后，可以进行不同的设置。图 1-17 可以配置 PyCharm 界面上的菜单和工具栏，

（3）配置菜单和工具栏

图 1-18 可以配置编辑器中代码的字体、大小、背景等个性化的配置方案。

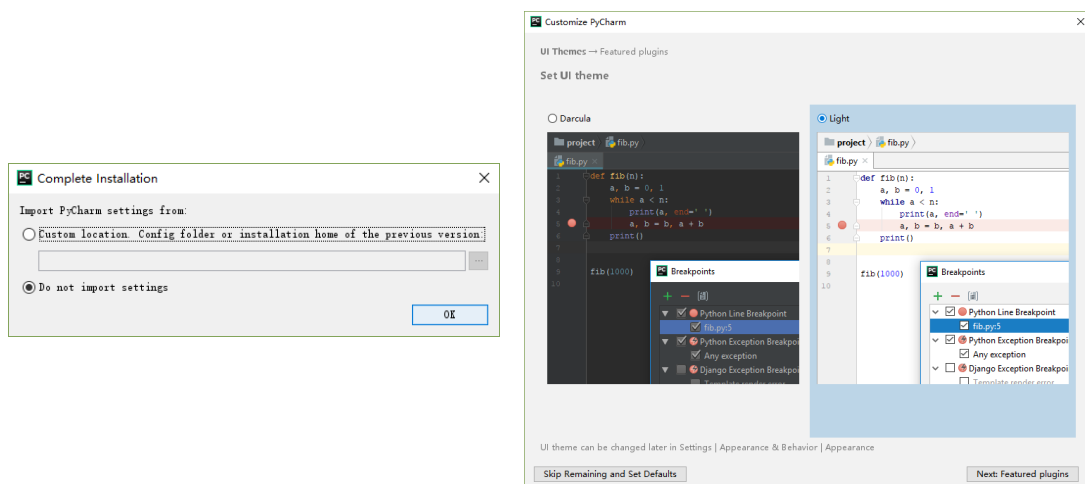


图 1-15 PyCharm 首次启动的配置选项

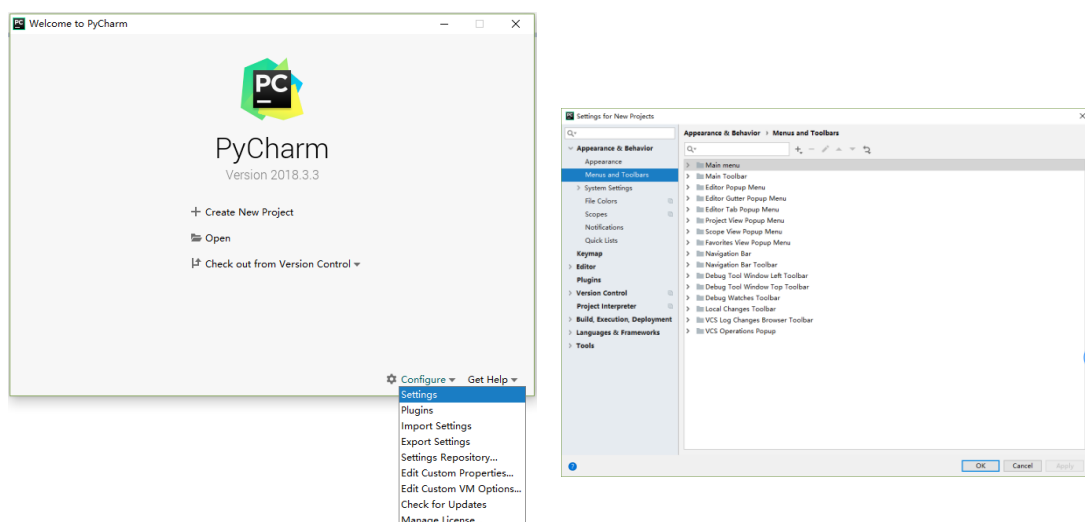


图 1-16 PyCharm 启动欢迎界面

图 1-17 配置菜单和工具栏

(4) 配置 PyCharm 的解释器

图 1-17 中选“Project Interpreter”可以设置默认的 PyCharm 的解释器，即设置在 PyCharm 中编辑的程序由哪个解释器解释后执行。点击下拉列表框中的“Show All”后进行设置。设置完成后类似图 1-19 所示。

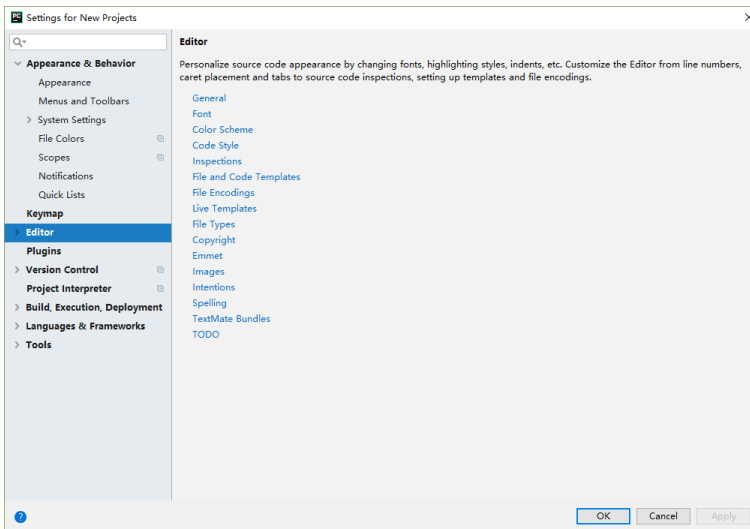


图 1-18 配置编辑器的个性化配色方案

(5) 添加扩展库

在图 1-20 的配置对话框中，选右侧的“+”会出现图 1-21 所示的对话框。输入需要安装的库的名称，单击“Install Package”按钮安装。

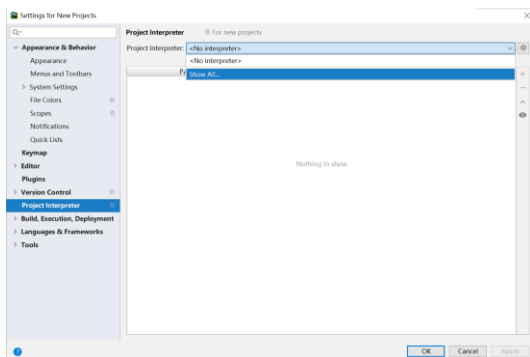


图 1-19 设置默认 PyCharm 的解释器（一）

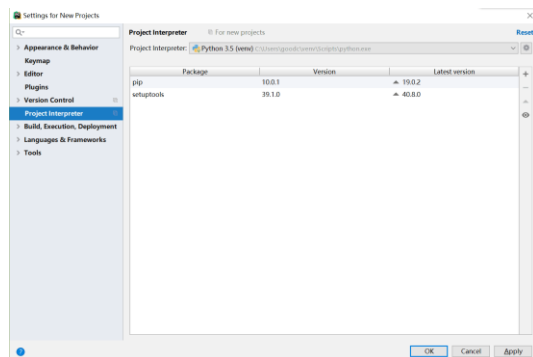


图 1-20 设置默认 PyCharm 的解释器（二）

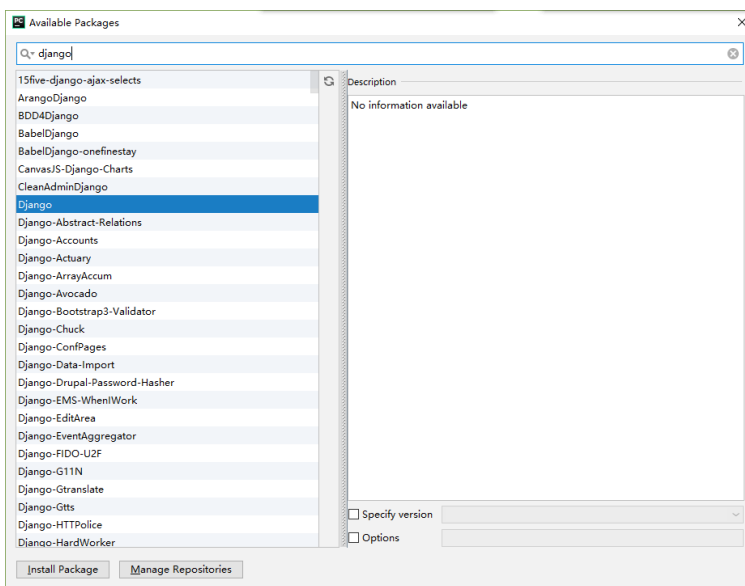


图 1-21 添加扩展库

1.3 编写并运行一个 Python 程序

1.3.1 交互式执行

在只安装了 Python 而未安装其他 IDE 的环境下，运行 Python 程序有两种方式：交互式 and 文件式。交互式是指 Python 的指令逐条输入并由 Python 解释器逐条执行，它有两种启动和运行方式。

第一种交互式方法：在开始菜单中单击 Python 图标“Python 3.5 (64-bit)”打开 Python 命令行解释器，如图 1-22 所示，在命令提示符>>>后输入如下 Python 指令：

```
print("Hello World")
```

按回车键 Enter 后显示输出结果“Hello World”。

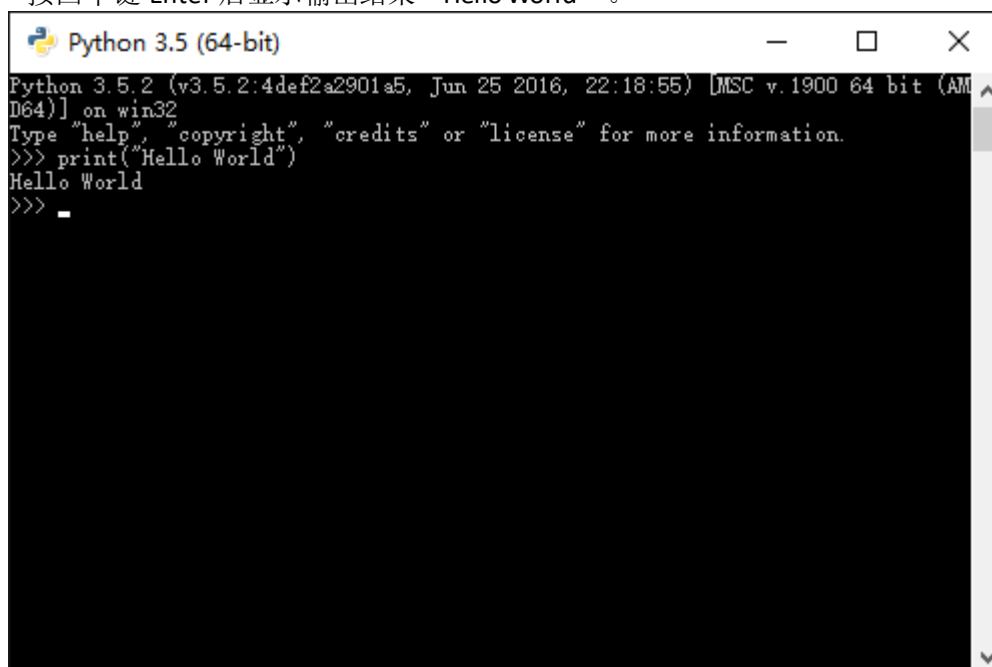


图 1-22 在 Python 命令行窗口中交互式执行 Python 代码

第二种交互式方法：在开始菜单中单击图标“IDLE (Python 3.5 64-bit)”打开 Python 自带的集成开发环境 IDLE 窗口，在命令提示符>>>后输入如下 Python 指令：

```
print("Hello World")
```

执行结果如图 1-23 所示。在命令提示符>>>后还可以使用 Alt+P 浏览历史命令中的上一条命令，或 Alt+N 浏览历史命令中的下一条命令。IDLE 中的一些常用快捷键列在表 1-1 中。

表 1-1 IDLE 中一些常用的快捷操作

快捷键	作用
Alt+/	自动补全代码（查找编辑器内已经写过的代码来补全）。
Ctrl+Shift+Space	补全代码提示。
Ctrl+]	增加当前行代码的缩进。
ctrl+[	减少当前行代码的缩进。
Alt+3	在当前行代码首字符前加注释标记##。
Alt+4	减少在当前行代码首字符开始处的注释标记##。
Ctrl+N	新建文件。
Ctrl+S	保存文件。
F5	运行文件。

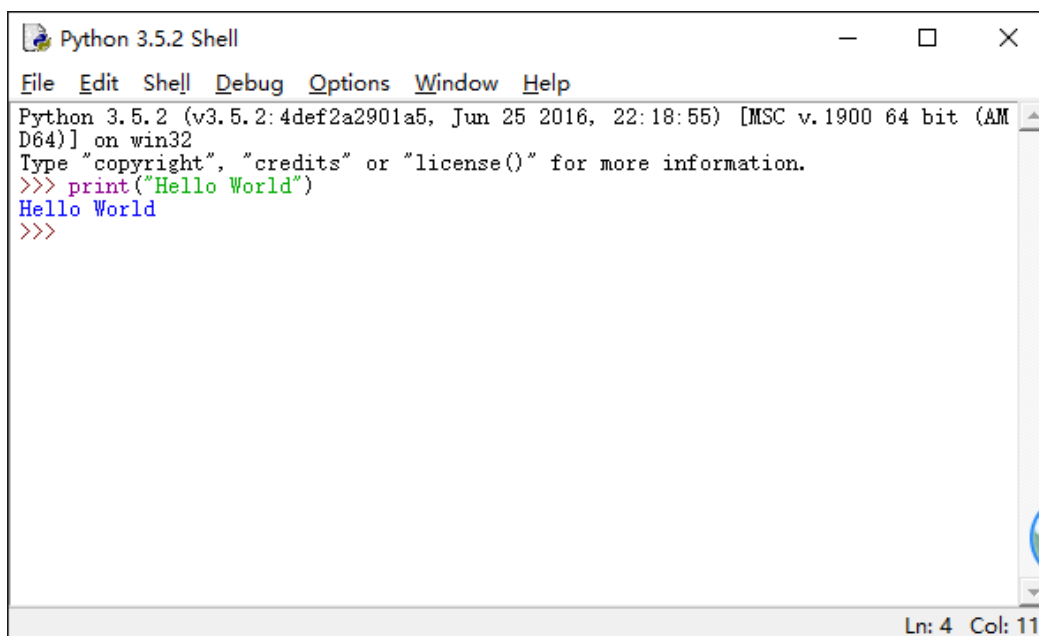


图 1-23 在 IDLE 中交互式执行 Python 代码

1.3.2 文件式执行

文件式 Python 程序是指将需要执行的多条语句存放在文件中。

(1) Python 文件的类型

Python 程序常用的文件类型有：

- 源代码文件：由 Python 程序解释，不需要编译，以 .py 为文件扩展名。本书的文件以此为主。
- 字节代码文件：是根据 py 源文件编译而成的二进制字节码文件，由 Python 加载执行，速度快，能够隐藏源代码，以 .pyc 为文件扩展名。在命令行解释界面下输入以下两条命令，可以将 .py 文件转换为 .pyc 文件。

```
>>>import py_compile
>>>py_compile.compile(文件名)
```

注意文件名必须加 .py 的扩展名，文件名前可以包含文件的存放路径。路径分隔字符“\”需要用转义字符或在文件名字符串前加 r 前缀。

- 优化代码文件：以 .pyo 为文件扩展名，是优化编译后的程序，也是二进制文件，适合用于嵌入式系统。在 Windows 的命令提示符窗口下，输入以下格式的命令，可以将 .py 源文件转换为 .pyo 文件。

```
python -O -m py_compile 文件名
```

注意文件名必须加 .py 的扩展名，文件名前可以包含文件的存放路径。

.py 文件转换后的 .pyc 文件和 .pyo 文件位于源文件目录下新增的 __pycache__ 中。

(2) 两种执行 Python 文件的方法

文件式 Python 程序也有两种运行方法。

第一种文件式执行方法：用其他的编辑软件，比如 Windows 自带的记事本，将编写好的程序以扩展名为 .py 的文件保存。我们预先将只有一行指令的 `print("Hello World")` 的程序存为 `hello.py`，存放在 d 盘的根目录下。

打开 Windows 的命令窗口，输入 “d:” 后按回车键，进入程序所在的 d 盘根目录，输入命令 “python hello.py” 或 “hello.py” 后按回车键，即可运行 `hello.py` 程序。如图 1-24

所示。

第二种文件式执行方法：打开 IDLE，在菜单中选择 File->New File 或按快捷键 Ctrl+N，打开一个新窗口。在文本编辑区输入 `print("Hello World")`，并保存为 `Hello.py` 文件。然后在菜单中选择 Run->Run Module 或按快捷键 F5，即可运行该文件，运行结果会显示在“Python 3.5.2 Shell”窗口中，如图 1-25 所示。

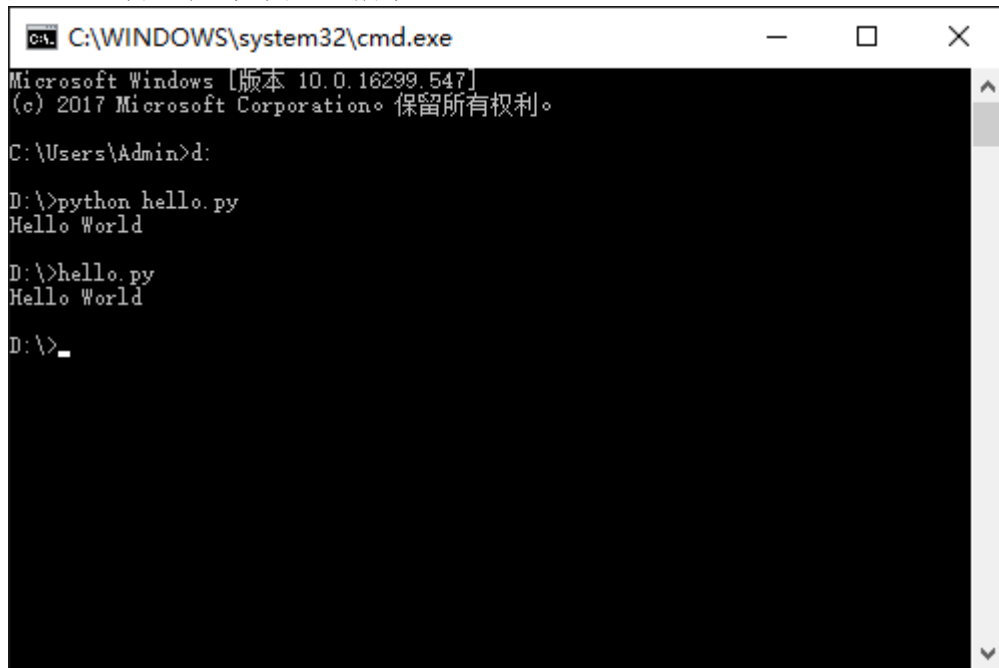


图 1-24 在 Windows 命令行窗口中运行 Python 程序文件

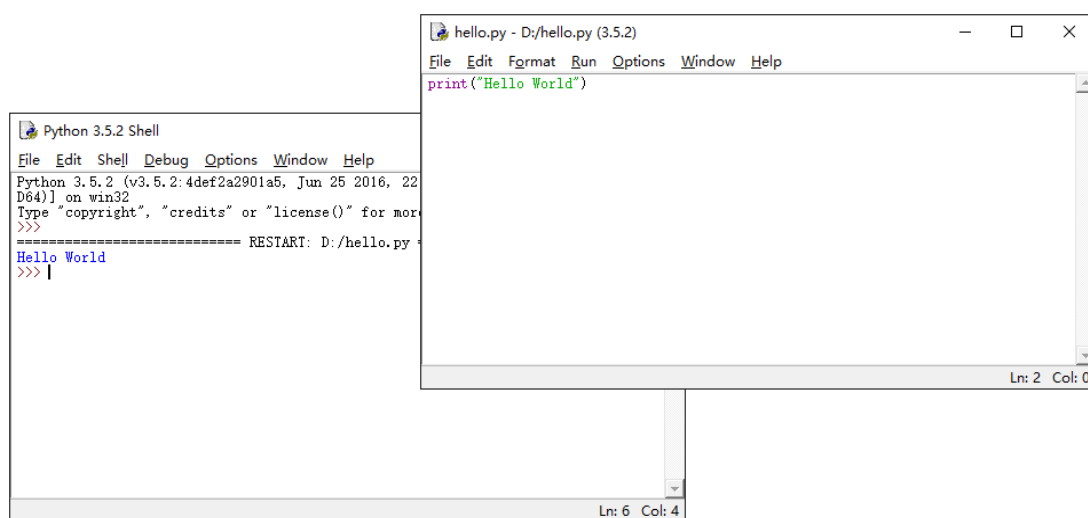


图 1-25 在 IDLE 窗口编辑程序并运行

1.3.3 使用 PyCharm 编写第一个程序

(1) 新建第一个程序

启动 PyCharm 后，会出现如图 1-26 所示的启动对话框，用户可以选择“Create New Project”来新建一个项目。

程序设计中的项目，是将你要解决的问题视为一个复杂工程，要解决问题就是要管理好

这个工程，即使这个所谓的工程可能只是需要一行代码解决。真正复杂的问题确实需要多行代码、多个文件才能解决，所有的代码、文件和辅助资料都是项目的组成部分。PyCharm 中的项目就是把解决问题的所有内容组织在一起。一个项目对应一个文件夹，即项目内的东西都会被组织在同一个文件夹内，并通过项目文件统一管理。

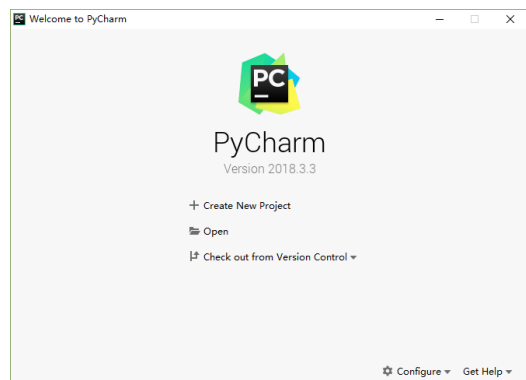


图 1-26 PyCharm 启动对话框

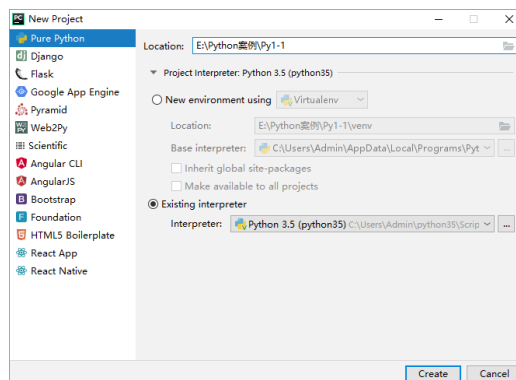


图 1-27 新建项目

选“Create New Project”后会弹出如图 1-27 所示的“New Project”对话框，用于设置新建项目的相关参数。“Location”是项目存放的文件夹位置和文件夹名，“Existing interpreter”是之前配置好的 Project 解释器。设置好这两项后，单击“Create”按钮，即可进入新建的 Python 项目管理窗口，如图 1-28 所示。左侧区域是当前项目内容的列表，右侧是正在编辑的项目内当前内容的显示区域。当前项目内没有任何内容。

选择主菜单 File->New，会弹出一个 New 的快捷菜单，如图 1-29 所示，选择“Python File”并输入文件名 Py1-1main.py。输入以下代码：

```
#1. name=input('请输入你的姓名: ')
#2. print('欢迎%s 来到 Python 的世界! '%name)
```

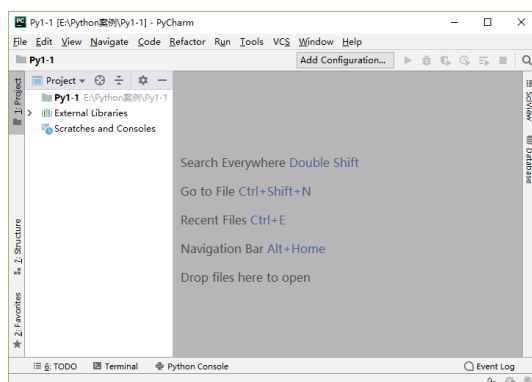


图 1-28 PyCharm 主界面

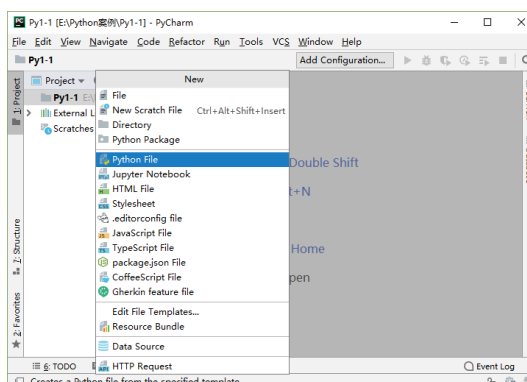


图 1-29 新建 Python 文件

注意，在上面的代码文本中，每一行前面的“#”符号及数字不是程序内容，是本书为了说明方便增加的，用户在录入编辑本书中的示例程序时只要输入点号后面的部分即可。本书会根据文字说明的需要部分示例代码前加上“#行号。”。

在左侧栏的 Py1-1main.py 上右击鼠标，选择“Run 'Py1-1main.py'”，如图 1-30 所示，在下方的执行区域输入姓名，最终的运行结果如图 1-31 所示。

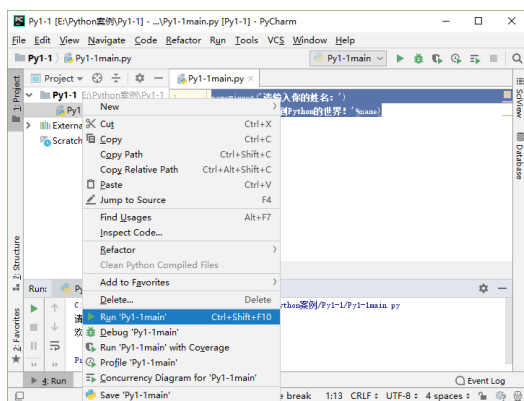


图 1-30 PyCharm 运行程序

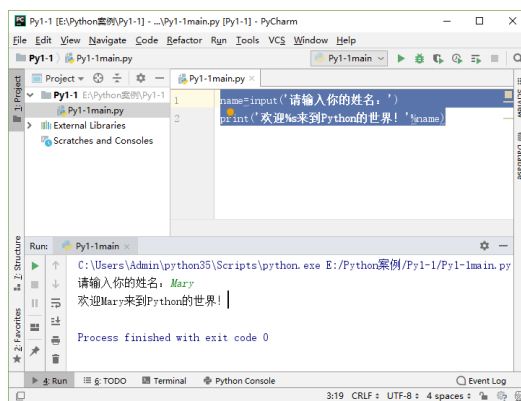


图 1-31 运行结果

1.4 安装扩展库的方法

1. 扩展库概述

早期，计算机领域的编程是通过调用官方提供的 API（Application Programming Interface，应用程序编程接口）来减少编程的工作量。API 调用能够直接使用系统已有的功能，自己不需要再重复编程。随着程序开发开源理念的出现、兴起和蓬勃发展，专业人员编写程序的难度降低了，他们将各领域最优秀的研究和开发成果以开源库形式发布出来。编程人员不需要再探究不同专业中每个具体算法的逻辑功能和设计，只需要利用扩展库（又称第三方库）进行代码复用即可。

现在程序开发更普遍的方式是利用可重用资源来快速构建应用。Python 拥有大量的扩展库，这些函数库覆盖了信息技术领域所有的发展方向。Python 的扩展库不一定是采用 Python 语言编写的，很多采用 C、C++ 等语言编写，这些库经过封装就能供 Python 语言程序调用。Python 语言具有类似胶水的粘性，围绕着它迅速形成了全球范围的编程语言开放社区。因此，Python 也被称为“胶水语言”，可以作为各类编程语言的接口。

Python 可复用的代码包括库（library）、模块（module）、类（class）和程序包（Package）等。这些可复用的代码就是编程时的“积木块”，也包括用户自己编写的可复用代码。编写 Python 程序，就是将这些积木块搭接在一起，形成自己需要的程序。

根据来源的不同，Python 内置的库称为标准库，其他库称为扩展库（或第三方库）。标准库在安装 Python 解释器后，就存在用户的计算机中了，只要用正确的步骤和方法就可以调用标准库中的代码。而扩展库则需要经过安装，才能出现在用户的计算机中，才能在 Python 程序中被引用。

2. 安装扩展库

（1）pip 工具安装

在 Python 中，最常用且高效的扩展库安装是通过包管理工具 pip 完成的，它是由官方提供并维护的安装工具。用户在安装 Python 解释器时，应该勾选“Add Python 3.5 to PATH”复选框。若未勾选，则可以重新运行安装程序来添加 pip。

pip 工具需要在 Windows 的命令提示符窗口下使用，而非在 Python 的 Python 命令行解释器或 IDLE 中使用。

打开 Windows 命令提示符窗口的方法：按 Win+R 打开“运行”对话框，如图 1-32 所示。在“打开”文本框中输入 cmd 命令，点击“确定”按钮后，即可打开 Windows 命令提示符窗口，如图 1-33 所示。

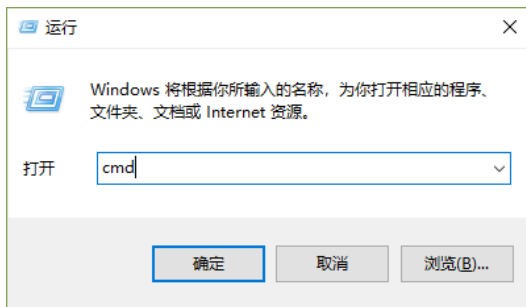


图 1-32 “运行”对话框

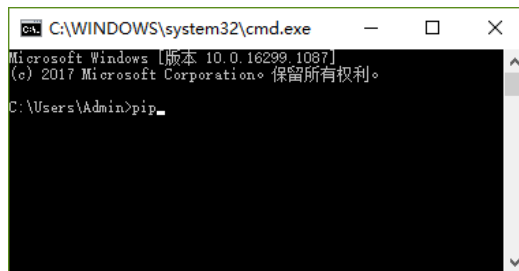


图 1-33 Windows 命令提示符窗口

在 Windows 命令提示符窗口中输入 `pip`，并按“回车”键确认，会显示 `pip` 命令的参数和含义。`pip` 支持安装、下载、卸载、列表、查看、查找等一系列安装和维护子命令。

`pip` 命令的基本格式如下：

`pip <command> [options]`

子项 `command` 和 `options` 可用的形式及含义见图 1-34。



图 1-34 `pip` 命令的参数和含义

使用 `pip` 工具进行扩展库管理主要有以下方式：

- 查看已安装的扩展库

```
pip list
```

- 安装扩展库

```
pip install <库名 1>[<库名 2>...]
```

表 1-2 列出了一些实用的扩展库及安装命令。

表 1-2 Python 扩展库

库名	用途	pip 安装指令
NumPy	矩阵运算	pip install numpy
Matplotlib	产品级 2D 图形绘制	pip install matplotlib
PIL	图像处理	pip install pillow
sklearn	机器学习和数据挖掘	pip install sklearn
Requests	HTTP 协议访问	pip install requests
Jieba	中文分词	pip install jieba
Beautiful Soup 或 bs4	HTML 和 XML 解析	pip install beautifulsoup4
Wheel	Python 文件打包	pip install wheel
Pyinstaller	打包 Python 源文件为可执行文件	pip install pyinstaller
Django	Python 最流行的 Web 开发框架	pip install django
Flask	轻量级 Web 开发框架	pip install flask
WeRoBot	微信机器人开发框架	pip install werobot
Networkx	复杂网络和图结构的建模和分析	pip install networkx
SymPy	数学符号计算	pip install sympy
pandas	高效数据分析	pip install pandas
PyQt5	基于 Qt 的专业级 GUI 开发框架	pip install pyqt5
PyOpenGL	多平台 OpenCV 开发接口	pip install pyopengl
PyPDF2	PDF 文件内容提取及处理	pip install pydbf2
Docopt	Python 命令行解析	pip install docopt
PyGame	简单小游戏开发框架	pip install pygame

- 更新已安装库的版本

```
pip install -U[库名]
```

参数-U 表示更新版本，库名是被更新扩展库的名称。

- 查询已安装扩展库的详细信息

查询一个库的命令格式：

```
pip show <库名>
```

例如以下命令可以查询 Pygame 扩展库。

```
pip show pygame
```

- 卸载已安装的扩展库

卸载一个库的命令格式：

```
pip uninstall <库名>
```

例如卸载 Pygame 扩展库的命令如下，输入命令后可能还需要用户再次确认卸载。

```
pip uninstall pygame
```

- 下载扩展库但不安装

下载一个库的命令格式：

```
pip download <库名>
```

例如以下命令可以下载 PyQt5 扩展库但不安装。

```
pip download PyQt5
```

pip 是 Python 扩展库最主要的安装方式，可以安装绝大部分扩展库。但是，因为一些历史、技术和政策等因素，有些扩展库无法用 pip 安装。此时就需要用其他方法安装了。

(2) 自定义安装

自定义安装是指按照扩展库提供方提供的步骤和方式安装。扩展库都有主页用于维护扩展库的代码和文档。用户可以打开扩展库的维护主页，自行下载扩展库的相关文档，并根据指示步骤安装。

当无法使用 pip 安装扩展库时，可以进行自定义安装。

(3) 文件安装

由于 Python 某些扩展库仅提供源代码，使用 pip 下载后无法在 Windows 系统下编译安装。美国加州大学提供了一个网页，帮助 Python 用户获得 Windows 可直接安装的第三方库文件，链接地址如下：

<http://www.lfd.uci.edu/~gohlke/pythonlibs/>

该地址列出了一批在 pip 安装中可能会出现问题的扩展库。用户下载时可以根据已装 Python 的版本和计算机的字长（32 位或 64 位）下载适用的.whl 文件。下载后使用 pip 命令的安装子命令进行安装。格式如下：

```
pip install <文件名>
```

此处的文件名应包含.whl 文件的存放路径及文件名。

1.5 习题

1. 根据自己的理解，简述什么是程序和程序设计，它们有什么意义。
2. 根据自己的理解，简述什么是程序设计语言，它有什么功能。
3. 根据自己的理解，简述什么是低级语言和高级语言，它们各有什么特点。
4. 除了本章中介绍的集成开发环境外，还有哪些使用度较高的 Python 开发软件，选择一到三款 Python 开发软件，进行安装和试用。
5. 根据自己的理解，简述 Python 程序的运行方式。
6. 静态语句采用_____方式执行，如 C 语言、Java 语言等；脚本语言采用_____方式执行，如 Python 语言、JavaScript 语言等。
7. Python 自带的集成开发工具是_____。
8. 什么是标准库和第三方库？以下库哪些是标准库，哪些是第三方库，并简述各库的作用。

Django	jieba	math	Matplotlib	NumPy
os	pandas	Pygame	Pyinstaller	PyQt
random	re	scrapy	time	timedate
tkinter	turtle	wordcloud		

9. 简述如何安装和管理扩展库？

第2章 Python 语言基础

从组成上讲，程序包括数据和对数据的操作两部分。数据是计算机程序能够处理的信息在计算机内的表现形式。用计算机解决一个实际问题时，必须先对该问题进行抽象，以恰当的方式来描述问题中的数据。程序中的数据涉及数据类型、各种类型数据的表示方法和运算规则。本章主要介绍 Python 的数据类型、常量和变量、基本运算和常用函数。

2.1 常量

先看一些常量的例子，例如 5 和 1.23 是数值，'Hello' 是字符串，True 和 False 代表对和错。这些数据，直接可以从其字面了解到它们的含义。在常规的场景中，数字 2 总是表示它约定俗成的含义，所以它就是一个常量，常量的值和含义不会被改变。常量根据其书写的格式，被 Python 自动判定为相应数据类型的对象，统一进行管理。

Python 中有一个特殊的常量 None，用于表示无数据。None 不是 False、0 或空字符串，它和任何其他数据类型的值比较是否相等，永远返回的是 False。None 有自己的数据类型 NoneType，而且 NoneType 数据类型仅有 None 这唯一的一个数据。

2.2 数据类型

程序中被处理的数据有多种类型。Python 中数据的类型可以分为两大类：一类是基本数据类型，另一类是复合数据类型。基本数据类型包括数值（numbers）、字符串（string）和布尔类型（bool）。复合数据类型一般包括列表（list）、元组（tuple）、字典（dictionary）和集合（set）。

2.2.1 数值类型

数值类型中又分为整数（int）、浮点数（float）和复数（complex）三种类型。

1. 整数

整数是可正可负，没有小数点和小数部分，也没有分数的一种表示数值的形式。例如：9、-1002345、0 都是整数。

Python 中，整数除了日常生活中的十进制表示法以外，也可以用其他进制表示。非十进制的整数需要加上不同的前缀，这些前缀的第一个符号都是数字 0。以下是不同进制表示的整数：

- （1）十进制整数：就是日常生活中整数的表示方式，如 19、-4、123456789012345 等。
- （2）十六进制整数：以 0X 或 0x 开头，如 0X23、0xFF、-0x9CB2 等。
- （3）八进制整数：以 00 或 0o 开头，第 2 个符号是英文字母 O 或 o，如 0o7、-001735、0o735 等。
- （4）二进制整数：以 0B 或 0b 开头，如 0B101、0b11111111、-0B101010 等。

Python 中，整数类型的数据对象没有数据位数的限制，只受可用内存大小的限制。

2. 浮点

浮点数就是带小数的数值，可以是正数或负数。例如：9.03、-1073.201 都是浮点数。在 Python 中，浮点数也可以用科学记数法表示，例如：1073.201 可以表示为 1.073201E3。科学记数法中的 E 或 e 表示 10 的幂。这里的 1.073201E3 就等同于 1.073201×10^3 。

3. 复数

数学中，把形如 $z=a+bi$ 的数称为复数，其中 a 称为实部，b 称为虚部，i 是虚数单位。

Python 中也按照数学中的规则表示复数，但是虚数单位用 `j` 表示，例如：`5+6j`、`-3.7-9.45j` 都是复数。

2.2.2 字符串

一个字符串是一组字符的有序序列。在 Python 中用单引号'或者双引号"或者三引号（'''或''''）括起来的文本就是字符串。例如：`'a'`、`"my"`、`"""hello"""`、`"123"`都是字符串。

本书将在第 6 章详细介绍字符串的相关知识，这里先简单说明一下 Python 字符串表示的基本注意事项：

- （1）单引号'或者双引号"或者三引号（'''或''''）被称为字符串的定界符。三引号是指连续的三个单引号'或三个双引号"。定界符总是成对出现的，它们本身不是字符串的一部分。
- （2）一对定界符之间的字符可以是除本次定界符之外的任意其它英文字符、数字字符、汉字、中英文标点符号等。如果字符串内容中含有定界符，那么就要选择其他定界符来将字符串括起来，例如，`"I'm OK"`中含有单引号'，所以整个字符串就应该使用双引号"作为定界符。示例如下：

```
>>> "I'm a student!"
"I'm a student!"
>>> '''你好'123'''
"你好'123"
>>> ''' abc'"""123'''    #字符 c 后输入了一个单引号和两个双引号
' abc\'"""123'
```

注意，系统显示最后一个字符串的值时，以单引号作为定界符，字符 `c` 后的一个单引号前加了一个转义符号`\`，然后是两个双引号。

（3）转义字符

并不是所有的字符都是可以很简单的输入并显示的，例如制表符、换行符等，所以字符串中除了直接出现的普通字符外，Python 语言还允许使用一种特殊形式的字符，称为转义字符。它是以反斜杠`\`开头的一个字符序列，采用指定形式来表示指定的特殊字符。例如，`'abc\n123'`中的`\n`表示换行符。示例如下：

```
>>> print('abc\n123')
abc
123
```

看到输出结果中 `abc` 与 `123` 之间换行了。

转义字符的意思是该字符被解释为另一种含义，不再表示本来的字符。Python 中的常用转义字符如表 2-1 所示。

表 2-1 转义字符及含义

字符形式	含义	ASCII 码
<code>\n</code>	换行，将当前位置移到下一行开头	10
<code>\t</code>	水平制表（跳到下一个 Tab 位置）	9
<code>\v</code>	垂直制表	11
<code>\b</code>	退格，将当前位置移到前一列	8
<code>\r</code>	回车，将当前位置移到本行开头	13
<code>\f</code>	换页，将当前位置移到下页开头	12
<code>\a</code>	响铃报警	7
<code>\\</code>	代表一个反斜杠字符 <code>\</code>	92

\'	代表一个单引号字符。	39
\"	代表一个双引号字符。	34
\ooo	代表三位 8 进制数对应的字符。如\101 表示字符 A。	
\xhh	代表两位 16 进制数对应的字符。如\x41 表示字符 A。	
\uhhhh	代表四位 16 进制数表示的 Unicode 字符。	

(4) 原始字符串

字符串的定界符和转义起始符号\的使用带来了新的麻烦。例如，要使用字符串 `a\n2`，此处的 `\n` 不是一个换行符号，而是两个符号 `\` 和 `n`，则需要表示为 `a\\n2`，且这样表示还容易误解。Python 对字符串提供了一个前缀控制符 `r` 或 `R`，来简化定界符和 `\` 在字符中的使用。例如：可以用 `r'a\n2'` 表示字符串 `a\n2`。带有前缀 `r` 的字符串称为原始字符串。示例如下：

```
>>> print(r'a\n2')
a\n2
>>> print('a\n2')
a
2
```

(5) 三引号内的字符串可以直接按回车键换行的，会形成一个 `\n` 字符。单引号或双引号内的字符也可以换行，在行末输入一个 `\` 就可以换下一行输入，但是不会形成 `\n` 字符。命令行末的 `\` 实为续行符（参考 3.1.4 节内容）。示例如下：

```
>>> '''12
34
56'''
'12\n34\n56'
>>> '12\
34'
'1234'
>>> '12          # 输入 2 后按回车键，会出现出错提示，缺少了第二个单引号'
SyntaxError: EOL while scanning string literal
```

2.2.3 布尔类型

布尔值通常用来判断条件是否成立，它和布尔代数中的含义一致。一个布尔值只可能是 `True` 和 `False` 中的一个，要么是 `True`，要么是 `False`。`True` 表示逻辑真，`False` 表示逻辑假。在 Python 中，标识符是区分大小写的，请读者注意 `True`、`False` 中的大小写。

在 Python 中，`None`、任何数值类型中的 `0`、空字符串 `''`、空元组 `()`、空列表 `[]`、空字典 `{}`、空集合等都被当作 `False`。注意，是“被当作”而不是“等于”。其他值等价于 `True`。

2.2.4 复合数据类型

被程序处理的数据中，有些数据是有相互联系或需要整体处理的，Python 提供了批量处理数据的方法——复合数据类型。一个复合数据类型的值是一个包含多个有特定关系的数据的集合。Python 的复合数据类型有列表、元组、字典和集合。

1. 列表

列表是写在方括号 `[]` 之间、用逗号分隔开的数据元素序列。列表中数据元素的数据类型可以不相同，也可以嵌套。列表是有序的，列表中的数据元素可重复，也可以被修改。列表中的数据元素通过序号引用。例如：

```
>>> [25, 'Hi', -9.34, 3+4j]
```

```
[25, 'Hi', -9.34, (3+4j)]
>>> x = [1, 2, 3]      # 列表 x 中三个数据元素的编号是 0、1、2
>>> x[2] = -100        # 修改下标为 2 的 x 列表中的数据元素的值
>>> x
[1, 2, -100]
>>> j = 2
>>> x[j]               # 引用下标为 j 的列表值
-100
```

2. 元组

元组是写在圆括号（）之间、用逗号隔开的元素序列。元组中的数据元素类型可以不同，数据元素值不能修改、可重复、可嵌套。元组是有序的对象集合。任何一组用逗号分隔的数据，会被系统默认为元组。例如：

```
>>> (10, 20, 30)
(10, 20, 30)
>>> s = 3 + 7, 9 // 2, 'Y', True
>>> s
(10, 4, 'Y', True)
>>> workday = ('周一', '周二', '周三', '周四', '周五')
>>> workday
('周一', '周二', '周三', '周四', '周五')
>>> workday[3] = 'Thursday' #元组中的数据元素值不能修改
Traceback (most recent call last):
  File "<pyshell#45>", line 1, in <module>
    workday[3] = 'Thursday'
TypeError: 'tuple' object does not support item assignment
```

3. 字典

字典是写在大括号{}之间、用逗号分隔的数据元素集合。字典中的每个数据元素由一对关键字和值组成，中间用冒号:连接。字典中的元素通过关键字来存取，字典中关键字不可重复，值可以重复，字典是无序的对象集合。例如：

```
>>> d = {'name': 'Mike', 'age': 12, 'sal': 5000} # 字典 d 中包含三个数据元素
>>> d
{'name': 'Mike', 'age': 12, 'sal': 5000}
>>> d['age']      # 显示字典 d 中关键字为 age 的值
12
>>> d['sal'] = 6000
>>> d
{'name': 'Mike', 'age': 12, 'sal': 6000}
>>> d['bonus'] = 3000 # 字典中无 bonus 关键字，本行语句使字典增加了一个新元素
>>> d
{'name': 'Mike', 'age': 12, 'sal': 6000, 'bonus': 3000}
```

4. 集合

集合是写在大括号{}之间、用逗号分隔的数据元素集合。集合中的数据元素是无序的、不重复的，集合的数据元素可以添加、删除。示例如下：

```
>>> s1 = {'Jim', 'Mike', 'Mary', 'Tom', 'Jim'}
>>> s1
{'Jim', 'Tom', 'Mary', 'Mike'}
```

以上仅仅是简单介绍了这些复合数据类型的基本概念，具体的使用方法、特点和作用等都会在本书的第 6、7、8 章中详细展开。

2.3 变量

介绍本节内容前，先来看这行代码：

```
>>> a = 3.012456
```

这行代码执行时，实则完成了三个事情（如图 2-1 所示）：

（1）创建了一个 `float` 类型的对象 `3.012456`；

（2）创建了一个名为 `a` 的变量（变量 `a` 其实也是一个对象）；

（3）将变量 `a` 与 `float` 类型的对象 `3.012456` 相连接，从变量 `a` 到对象 `3.012456` 的连接称之为引用（变量 `a` 引用 `float` 类型对象 `3.012456`）。也可以沿用传统编程语言的說法：`a` 被赋值为 `3.012456`。

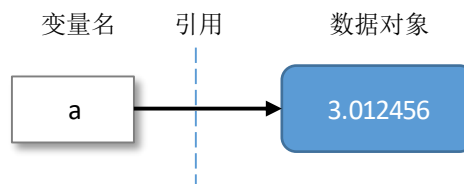


图 2-1 变量的赋值

1. 对象

Python 是面向对象的程序设计语言，它全面支持面向对象的程序设计思想。甚至传统的程序语言的基本语法元素在 Python 中也对象化了。

在面向对象的程序设计思想中，所谓的对象是指将一组数据和与这组数据有关的操作组装在一起，形成的一个实体。在语句“`a = 3.012456`”中，`a` 是一个对象，它包含了对数据 `3.012456` 的引用和 `a` 这个名字。`3.012456` 也是一个对象，它包含了它的数据类型 `float` 和值 `3.012456` 作为这个对象的数据。在 Python 内部实现中，`a` 和 `3.012456` 这两个对象还有其他的数据和操作。

面向对象程序中有“类”这个概念。所谓类，就是相同或相似性质的对象的抽象。对象的抽象是类，类的具体化就是对象。例如，`3` 和 `-90` 都是整数，整数是一个类，而 `3` 和 `-90` 则是具体的整数对象。在前一节中介绍的数据类型，也是各个不同的类。

Python 中，数据对象分为两种：可变数据对象和不可变数据对象。不可变数据对象包括整数 `int`、浮点数 `float`、复数 `complex`、字符串 `str`、元组 `tuple` 等，可变数据对象包括列表 `list`、集合 `set`、字典 `dict` 等。

2. 标识符与关键字

Python 标识符用来标识一个变量、函数、类、模块或其他对象的名称。Python 中标识符命名应遵循如下规则：

- （1）标识符中可用的符号有英文字母、汉字、数字或下划线；
- （2）第 1 个字符不能是数字；
- （3）英文字母大写和小写是有区别的，即对大小写敏感；

（4）Python 有一些标识符有固定含义和作用，被称为关键字或保留字，列在表 2-2 中。关键字不能用做变量名、函数名或类名等。查看 Python 关键字的方法如下：

```
>>> import keyword
>>> print(keyword.kwlist)          # 查看所有 Python 关键字
['False', 'None', 'True', 'and', 'as', 'assert', 'break', 'class', 'continue', 'def', 'del', 'elif', 'else',
```

'except', 'finally', 'for', 'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']

表 2-2 Python3.5 中的关键字

and	def	finally	in	not	True
as	del	for	is	or	try
assert	elif	from	lambda	pass	while
break	else	global	None	raise	with
class	except	if	nonlocal	return	yield
continue	False	import			

3. 变量

Python 中，变量的作用是对对象内存空间的引用。数据对象是系统分配的一块足够大的内存，用于存放该对象的数据和与之相关的操作。变量对对象的引用可以看作是从变量到对象的指针。当代码中出现变量名时，系统沿着变量的引用指针找到数据，并使用该数据。因此，在代码中使用变量名就是使用被指向的数据，多数情况下可以直接理解为变量就是代表对应的数据。变量的引用是能改变的，即变量可变。

Python 的数据类型分为不可变数据类型和可变数据类型。Python 内部对它们的管理方法是不同的。

● 不可变数据类型

变量去引用一个不可变数据类型值时，是变量与数据之间的引用关系发生了改变。虽然原来的引用关系消失了，但原来的数据不一定消失。整数 `int`、实数 `float`、复数 `complex`、字符串 `string` 和元组 `tuple` 是不可变数据类型。不可变的数据类型约等于是可哈希的。

变量通过赋值改变数据的示例如图 2-2 所示。图中左边方框中是执行的语句和显示结果，右边是变量引用的关系示意。

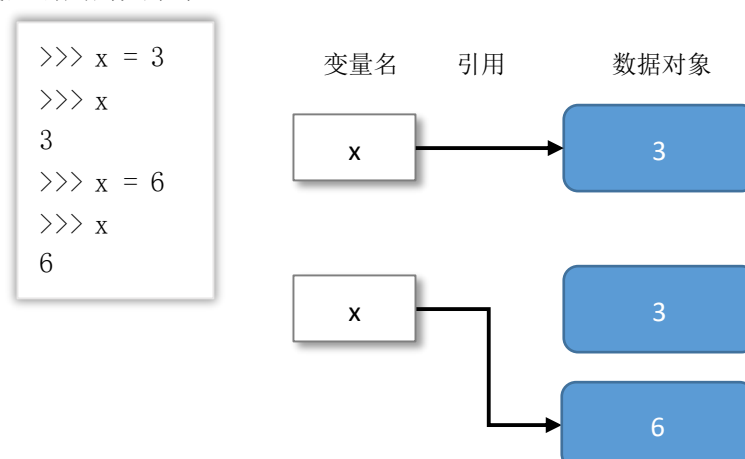


图 2-2 变量引用的可变

变量 `x` 第一次赋值后，指向并引用整型数据 `3`。第二次赋值后，指向并引用整型数据 `6`，原来 `x` 与 `3` 之间的引用关系消失了。若没有别的对象引用整数 `3`，则 `3` 占用的存储空间就会等待系统回收。

● 可变数据类型

通过变量改变可变数据类型的值，它们之间的引用关系不变，但存储数据的内存空间里的东西或内存空间大小会发生变化。列表 `list`、字典 `dict` 和集合 `set` 都是可变数据类型。

引用可变类型数据对象时，有些操作会令引用数据的内存发生改变，而有些操作则是在

此对象原有的内存空间上增加或减少，也就是它的内存地址会保持不变，但空间区域会变长或者变短。详情见 3.2.2 节。

由上述变量和数据的引用关系可知，变量是没有类型的，被变量引用的数据是有类型的。

- 单独的下划线（_）

单独的下划线（_）是一个特殊变量，用于表示上一次运算的结果。例如：

```
>>> 100
100
>>> _ + 20
120
```

4. id 函数

函数格式：

`id(obj)`

作用：返回 `obj` 对象的内存地址。示例如下：

```
>>> x = 3
>>> x
3
>>> id(x)
1618018832
>>> id(3)
1618018832
>>> x = 6
>>> x
6
>>> id(x)
1618018928
```

可以通过 `id` 函数来了解和理解变量和值之间的关系。

2.4 基本运算

计算机的大量功能都是通过各种各样的运算来完成的，为了完成这些运算，Python 提供了丰富的运算符（operator），这些运算符通过对数据进行处理来完成各种运算功能。由运算符、操作对象构成的式子被称为表达式（expression）。表达式是有值的，这个值就是运算符对各种数据进行处理的结果。

Python 支持算术运算符、关系运算符、逻辑运算符、位运算、成员运算符、身份运算符等。表达式中允许出现不同种类的运算，不同的运算符的优先级是不同的。本节先介绍算术运算符和位运算，其他运算在后续章节中陆续介绍。

2.4.1 算术运算

Python 的基本算术运算符有以下几种：

- + 正值运算符
- 负值运算符
- + 加法运算符
- 减法运算符
- * 乘法运算符
- / 实除运算符

// 整除运算符
% 求余运算符
** 乘方运算符

1. +正值、-负值、+加法、-减法、*乘法

这些运算符和数学中的使用习惯基本相同。对于乘法，要注意在表示 a 与 b 相乘时不能写为 ab ，而应该写做 $a*b$ ，即符号 $*$ 不能省略。示例如下：

```
>>> b = 100
>>> a = -b
>>> a
-100
>>> a = +b
>>> a
100
>>> a + b
200
>>> a - b
0
>>> a * 2
200
```

2. /实除、//整除和%求余

这几个运算符都要进行除法运算，但是所得的结果有所不同。

- /实除后一定得到一个浮点数结果，即使被除数和除数都是整数，也得到浮点数的商。示例如下：

```
>>> 4 / 7
0.5714285714285714
>>> 6.1 / 0.5, 6 / 3
(12.2, 2.0)
>>> 6 / 0.5, 6.1 / 5
(12.0, 1.22)
```

- //整除后获得除法运算结果中商的整数部分。这两个操作数都可以是整数也可以是实数。示例如下：

```
>>> 4 // 7
0
>>> 6.1 // 0.5, 6 // 3
(12.0, 2)
>>> 6 // 0.5, 6.1 // 5
(12.0, 1.0)
```

- %求余获得除法运算结果中的余数部分。两个操作数可以是整数也可以是实数。示例如下：

```
>>> 4 % 7
4
>>> 6.1 % 0.5, 6 % 3
(0.09999999999999964, 0)
```

其中， $6.1 \% 0.5$ 的求余应该是 0.1，但是因为计算机内的数据是二进制存储的，转换为十进制后，就会有误差产生。

```
>>> 6 % 0.5, 6.1 % 5
(0.0, 1.0999999999999996)
>>> 6 % 5, -6 % -5
(1, -1)
>>> -6 % 5, 6 % -5
(4, -4)
>>> 7.1 % 3, -7.1 % -3
(1.0999999999999996, -1.0999999999999996)
>>> -7.1 % 3, 7.1 % -3
(1.9000000000000004, -1.9000000000000004)
```

求余运算的结果一定小于除数的绝对值，并与除数同号。

3. **乘方

**运算符实现乘方运算，其优先级是最高的，仅次于括号()。

```
>>> -2 ** 4
-16
>>> 2 * 3 / 3 ** 2
0.6666666666666666
```

4. 浮点数的计算误差与相等判断

计算机内的所有数据是二进制形式存储的，在二进制和十进制相互转换后，就会有很微小的误差产生。Python 内的数值可能比理论真值略大或略小。例如：

```
>>> x = 2.3
>>> x - 1.3
0.9999999999999998
>>> x - 1.3 == 1      # 运算符==用于判断两个数是否相等
False                 # 结果为 False，表示 x-1.3 不等于 1
>>> -7.1 % 3
1.9000000000000004
```

运算符==用于判断两个数是否相等，因为进制转换会带来误差，直接用==判断两个实数结果是否相等，可能会得出错误的结果。

在 Python 中，要判断实数 a 和 b 是否相等的方法有：

(1) 可以看 $a - b$ 后的绝对值是否小于一个极小值。例如：

```
>>> a = 1.3
>>> b = 2.3 - 1
>>> abs(a-b) < 1e-10      # 内置函数 abs 求参数的绝对值
True
```

(2) 使用 math 库中的 isclose() 函数。

```
>>> import math
>>> 1.11 + 1
2.1100000000000003
>>> math.isclose(2.11, 1.11 + 1)
True
```

2.4. 2位运算*

Python 的位运算符号有以下几种：

& 按位与
| 按位或
^ 按位异或
~ 按位取反
<< 按位左移
>> 按位右移

Python 的位运算只能针对整数进行，并且是对整数的二进制逐位进行运算，运算结果也是一个整数。

先看以下代码：

```
>>> 17 & 135, 8 & 209, 97 & -7, -67 & 97
(1, 0, 97, 33)
```

执行结果用整数的知识根本无法理解。在开始介绍位运算前，先学习内置函数 `bin()`。

1. bin 函数

`bin` 函数可以查看整数的二进制。但计算机内的整数是用补码表示的，而用 `bin` 查看到的二进制码都是整数的原码值，且正数不显示符号位，负数用符号位-表示负值。例如：

```
>>> bin(17), bin(135), bin(-7), bin(-67)          # Python 不能显示负数的补码形态
('0b10001', '0b10000111', '-0b111', '-0b1000011')
```

但是我们可以巧用全 1 的二进制与负数按位与来查看负数的补码值。例如：

```
>>> c = -67          # -67 的 16 位补码是 1111111110111101
>>> bin(c & 0b1111111111111111)
'0b1111111110111101'
```

2. & 按位与

按位与运算符 `&`，对参与运算的两个值，如果两数二进制形式的相应位都为 1，则该位的结果为 1，否则为 0。例如：

```
>>> a = 17          # 17 的 8 位二进制补码是 00010001
>>> bin(a)
'0b10001'
>>> b = 135         # 135 的 8 位二进制补码是 10000111
>>> bin(b)
'0b10000111'
>>> a & b           # 1 的 8 位二进制补码是 00000001
1
>>> allone = 0b11111111    # 255 的 8 位二进制补码是 11111111
>>> a = -67          # -67 的 8 位二进制补码是 10111101
>>> bin(a & allone)
'0b10111101'
>>> b = 97           # 97 的 8 位二进制补码是 01100001
>>> bin(b)
'0b1100001'
>>> c = a & b        # a & b 的 8 位二进制补码是 00100001
```

```
>>> c
33
>>> bin(c & allone)
'0b100001'
```

3. | 按位或

按位或运算符|，对参与运算的两个值，如果两数二进制形式的相应位的两个二进制位有一个为 1 时，结果位就为 1。例如：

```
>>> a = -67    # -67 的 8 位二进制补码是 10111101
>>> b = 97     # 97 的 8 位二进制补码是 01100001
>>> c = a | b
>>> c          # -3 的 8 位二进制补码是 11111101
-3
>>> allone = 0b11111111
>>> bin(c & allone)
'0b11111101'
```

4. ^ 按位异或

按位异或运算符^，对参与运算的两个值，如果两数二进制形式的相应位的两个二进制位相异时，结果为 1。例如：

```
>>> a = -67    # -67 的 8 位二进制补码是 10111101
>>> b = 97     # 97 的 8 位二进制补码是 01100001
>>> c = a ^ b
>>> c          # -36 的 8 位二进制补码是 11111101
-36
>>> allone = 0b11111111
>>> bin(c & allone)
'0b11011100'
```

5. ~ 按位取反

按位取反运算符~，对运算数的每个二进制位取反，即把 1 变为 0，把 0 变为 1。例如：

```
>>> a = -67    # -67 的 8 位二进制补码是 10111101
>>> c = ~a
>>> c          # 66 的 8 位二进制补码是 01000010
66
>>> bin(c & allone)
'0b1000010'
```

6. << 按位左移

左移运算符<<，将第一个运算数的全部二进制位左移若干位，移动的位数由“<<”右边的数指定，低位则补 0。例如：

```
>>> a = 17     # 17 的 8 位二进制补码是 00010001
>>> c = a << 2
>>> c          # 68 的 8 位二进制补码 01000100
68
>>> bin(c & allone)
```

```
'0b1000100'
```

7. >> 按位右移

右移运算符>>, 将第一个运算数的全部二进位右移若干位, 移动的位数由“>>”右边的数指定。移出到小数点右侧的二进位丢弃, 正数高位补 0, 负数高位补 1。例如:

```
>>> a = 17    # 17 的 8 位二进制补码是 0001 0001
>>> c = a >> 2
>>> c          # 4 的 8 位二进制补码 0000 0100
4
>>> bin(c & allone)
'0b100'
```

2.4.3 运算优先级

当一个表达式中包含多个运算时, 运算的顺序不单纯是从左向右逐项计算的。例如: $3 + 4 + 5$, 是按从左向右逐项计算, 而 $3 + 4 * 5$, 是先计算*后计算+。故运算符是有优先级的, 优先级高的先计算, 优先级低的后计算。圆括号()的优先级最高, 即先计算圆括号内的部分。例如, $(3 + 4) * 5$, 是先计算+后计算*。Python 中运算符的优先级由高到低逐级递减, 列在表 2-3 中。

表 2-3 Python 运算符的优先级

运算符	说明
**	乘方（最高优先级）。
~ + -	按位翻转，一元加号和减号。
* / % //	乘，除，取模，取整除。
+ -	加法，减法。
>> <<	右移，左移运算符。
&	按位与。
^	按位异或。
	按位或。
is notis in notin	身份运算符，成员运算符。
<= < > >= == !=	比较运算符。
not	逻辑非。
and	逻辑与。
or	逻辑或。

2.5 常用系统函数

2.5.1 常用内置函数

内置函数包含在模块 `builtins` 中, 每次启动 Python 解释器时都会自动将该模块装入内存中, 内置函数可以直接调用, 不需要使用 `import` 命令引用库。

表 2-4 Python3.5 的内置函数表

<code>abs()</code>	<code>all()</code>	<code>any()</code>	<code>ascii()</code>	<code>bin()</code>
<code>bool()</code>	<code>bytearray()</code>	<code>bytes()</code>	<code>callable()</code>	<code>chr()</code>
<code>classmethod()</code>	<code>compile()</code>	<code>complex()</code>	<code>copyright()</code>	<code>credits()</code>
<code>delattr()</code>	<code>dict()</code>	<code>dir()</code>	<code>divmod()</code>	<code>enumerate()</code>

eval()	exec()	exit()	filter()	float()
format()	frozenset()	getattr()	globals()	hasattr()
hash()	help()	hex()	id()	input()
int()	isinstance()	issubclass()	iter()	len()
license()	list()	locals()	map()	max()
memoryview()	min()	next()	object()	oct()
open()	ord()	pow()	print()	property()
quit()	range()	repr()	reversed()	round()
set()	setattr()	slice()	sorted()	staticmethod()
str()	sum()	super()	tuple()	type()
vars()	zip()			

这些内置函数可以分成以下类别：

1. 数学函数

表 2-5 内置的数学函数

函数	说明
abs(x)	返回 x 的绝对值。
divmod(x, y)	返回一个包含商和余数的元组(x // y, x % y)。
pow(x, y, z=None,)	返回 x ** y (x 的 y 次方) 或 x ** y % z。
round(number[, ndigits])	返回浮点数 number 的 ndigits 位小数的四舍五入值，单参数时四舍五入为整数。

2. 数据类型转换函数

表 2-6 内置的数据转换函数

函数	说明
ascii(obj)	返回一个表示对象 obj 的字符串，但是对于字符串中的非 ASCII 字符则返回通过 repr()函数使用\x, \u 或\U 编码的字符。
chr(x)	返回 Unicode 码值为 x 的字符，0 <= x <= 0x10ffff。
bin(x)	将十进制整数转换成二进制整数，返回结果为字符串。
hex(x)	将十进制整数转换成十六进制整数，返回结果为字符串。
oct(x)	将十进制整数转换成八进制整数，返回结果为字符串。
complex(real[, imag])	返回值为 real+imag*j 的复数。
float([x])	将整数或字符串 x 转换为浮点数，无参时返回 0.0。
int(x)	将数字字符串 x 转换为整型。
bool([x])	将参数 x 转换为布尔类型，无参则返回 False。
dict([x])	将参数 x 转换为字典，无参则创建空字典。
list([x])	将参数 x 转换为列表，无参则创建空列表。
set([x])	将参数 x 转换为集合，无参则创建空集合。
str([x])	将参数 x 直接转换为字符串，无参则创建空串。
tuple([x])	将参数 x 转换为元组，无参则创建空元组。

2.5. 2常用库函数

1. 引用库

除了内置函数外，Python 还有随编译环境一起安装的可复用代码，即标准库。标准库里往往包含常量、类和函数，在用户程序中，导入库即可使用它们。此外，Python 社区中还有数量众多的可复用代码，即第三方库，用户可以下载安装并在自己的程序中引用它们。

引用标准库或安装好的第三方库中的内容之前，必须使用以下格式之一的语句导入库。

(1) import 语句

import 语句的一般格式：

```
import 模块 1[as 别名 1][, 模块 2[as 别名 2][... 模块 N[as 别名 N]]
```

此后，调用被引用模块中的函数时，引用格式如下：

格式 1：模块名.函数名(参数)

格式 2：别名.函数名(参数)

例如：引用内置模块 time 和 random，并调用函数的代码如下：

```
>>> import time, random
>>> time.sleep(3)          # 程序休眠 3 秒
>>> r.randint(1, 10)       # 生成一个 1 到 10 之间的随机整数
```

(2) from...import 语句

from...import 语句的一般格式：

```
from 模块 import 函数名 1[as 别名 1][... 函数名 N[as 别名 N]]
```

该命令只导入库中的指定函数，“模块”是要引用的库名。导入后可按以下格式直接调用函数：

格式 1：函数名(参数)

格式 2：别名(参数)

例如：前述调用模块 time 的 sleep()函数和模块 random 的 randint()函数也可写为：

```
>>> from time import sleep
>>> from random import randint as ri
>>> sleep(3)
>>> ri(1,10)
```

(3) from...import * 语句

from 模块 import *

该语句导入指定模块里的所有函数，导入后可直接调用库中的所有函数。例如：

```
>>> from time import *
>>> sleep(3)
```

2. 常用标准库函数

(1) math 库中的常用函数

表 2-7 math 库中的常用函数及常量

常量或函数	说明
math.pi	圆周率常量，值为 3.141592653589793。
math.e	自然对数常量，值为 2.718281828459045。
math.inf	正无穷大常量，负无穷大为-math.inf。
math.nan	非浮点数标记常量，NaN（Not a Number）。
math.fabs(x)	返回 x 的绝对值。
math.fmod(x, y)	返回 x 与 y 的模。
math.fsum([x, y,...])	浮点数精确求和。
math.ceil(x)	向上取整，返回不小于 x 的最小整数。

math.floor(x)	向下取整，返回不大于 x 的最大整数。
math.factorial(x)	返回 x 的阶乘，如果 x 是小数或负数，返回 ValueError 出错。
math.gcd(a, b)	返回 a 与 b 的最大公约数。
math.modf(x)	返回浮点数 x 的小数和整数部分。
math.trunc(x)	返回浮点数 x 的整数部分。
math.isclose(a, b)	比较 a 和 b 的相似性，返回 True 或 False。
math.pow(x, y)	返回 x 的 y 次幂。
math.exp(x)	返回 e 的 x 次幂，e 是自然对数。
math.sqrt(x)	返回 x 的平方根。
math.log(x[, base])	返回 x 的 base 对数值。
math.log2(x)	返回 x 的 2 对数值。
math.log10(x)	返回 x 的 10 对数值。
math.sin(x)	返回 x 的正弦函数值，x 是弧度值。
math.cos(x)	返回 x 的余弦函数值，x 是弧度值。
math.tan(x)	返回 x 的正切函数值，x 是弧度值。
math.asin(x)	返回 x 的反正弦函数值，x 是弧度值。
math.acos(x)	返回 x 的反余弦函数值，x 是弧度值。
math.atan(x)	返回 x 的反正切函数值，x 是弧度值。

(2) random 库中的常用函数

表 2-8 random 库中的常用函数

函数	说明
seed(a=None)	初始化随机数种子，默认值为当前系统时间。
random()	生成一个[0.0, 1.0)之间的随机小数。
randint(a, b)	生成一个[a, b]之间的整数。
choice(seq)	从序列类型（例如列表）中随机返回一个元素。
shuffle(seq)	将序列类型中元素随机排列，返回打乱后的序列。
sample(pop, k)	从序列类型 pop 中随机选取 k 个元素，以列表类型返回。

(3) time 库中的常用函数

表 2-9 time 库中的常用函数

函数	说明
time.asctime([tupletime])	接受时间元组并返回一个可读的形式为"Fri Oct 11 18:07:14 2019"（2019 年 10 月 11 日 周五 18 时 07 分 14 秒）的 24 个字符的字符串。
time.clock()	用以浮点数计算的秒数返回当前的 CPU 时间。用来衡量不同程序的耗时，比 time.time() 更有用。
time.gmtime([secs])	接收时间戳并返回格林威治天文时间下的时间元组 t，可选的参数 secs 表示从 1970-1-1 以来的秒数。
time.mktime(tupletime)	与 time.gmtime 是相反的操作，接受时间元组，返回时间戳（1970-1-1 以来的浮点秒数）。
time.sleep(secs)	推迟调用线程的运行，secs 指秒数。
time.time()	返回当前时间的时间戳（1970-1-1 以来的浮点秒数）。

(4) datetime 库中的常用函数。

表 2-10 datetime 库中的常用函数及常量

常量或函数	说明
MINYEAR	常量，值为 1，datetime 所能表示的最小年份。
MAXYEAR	常量，值为 9999，datetime 所能表示的最大年份。
datetime.today()	返回一个表示当前本地时间的 datetime 对象。
datetime.now([tz])	返回一个表示当前本地时间的 datetime 对象，如果提供了参数 tz，则获取 tz 参数所指时区的本地时间。
datetime.fromtimestamp(timestamp[, tz])	根据时间戳创建一个 datetime 对象，参数 tz 指定时区信息。

3. 序列结构或可迭代数据的函数

表 2-11 内置的序列结构或可迭代数据函数

函数	说明
all(x)	判断给定的序列或可迭代参数 x 中的所有元素的值是否都等价于 True，如果是返回 True。
any(x)	判断给定的序列或可迭代参数 x 中的所有元素的值是否都等价于 False，如果是返回 True。
enumerate(x[, start])	将序列或可迭代参数 x 中的所有元素的序号和元素值组合成若干元组，返回这些元组，其中元素序号从 0 或参数 start 开始。
len(x)	返回给定的序列或可迭代参数 x 的长度或元素个数。
max(x)	对给定的序列或可迭代参数 x，返回最大的元素值。
min(x)	对给定的序列或可迭代参数 x，返回最小的元素值。
next(x)	返回可迭代参数 x 的下一个元素。
range(stop) range(start, stop[, step])	返回 range 对象，其中包含左闭右开区间[start,stop)内以 step 为步长的整数。
reversed(x)	返回参数 x 逆序后的序列或可迭代对象。
sorted(x, key=None, reverse=False)	返回排序后的列表，参数 x 可以为序列或可迭代对象。
sum(x, start=0)	返回序列 x 中所有元素之和，若有 start，则返回 start+所有和。
zip(iter1[, iter2 [...]])	参数 iter1、iter2...为序列值或可迭代对象，返回 zip 对象，zip 对象元素为(iter1[, iter2 [...]])的元组，结果元素的个数取决于参数元素个数最少的值。
map(func, *iterables)	将单参数函数的函数名做为 func 参数，参数*iterables 为序列或可迭代对象，作用是将每个元素传递给单参数处理，得到各项函数值，并以 map 对象返回。
filter(func or None, iterable)	将单参数函数的函数名做为 func 参数，参数 iterables 为序列或可迭代对象，作用是将每个元素传递给单参数处理，将返回函数值为 True 的原序列值或可迭代对象，若首参数为 None，则返回值等价于 True 的元素值。

4. 其他函数

表 2-12 内置的其他函数

函数	说明
dir([object])	不带参数时，返回当前范围内的变量、方法和定义的类型列表；带参数时，返回参数的属性、方法列表。
eval(s, g=None, l=None)	执行字符串 s 中表达式或代码，并返回表达式的值。
exit([n])	终止 Python 程序，参数 n 表示程序退出类型，通常情况下 0 表示程序

	正常退出。
help(x)	用于查看函数或模块用途的详细说明。
id(x)	获取对象 x 的标识（即内存地址）。
input(prompt=None)	从标准输入设备读入一个字符串，返回结果是字符串类型。
isinstance(x, class_or_tuple)	返回 x 是否为某数据类型，或 x 是否为类的实例，参数 class_or_tuple 可以是元组。
open()	用于打开一个文件，创建一个 file 对象。
print()	输出信息到屏幕或指定设备。
type(object)	返回 object 的类型。

2.6 Python 的帮助系统

对于初学者来说，Python 语法学习只靠一本书是不够用的，利用 Python 的帮助系统就是一个快捷准确的途径。

1. dir()函数和 help()函数

获取 Python 帮助的一种方法是使用内置 dir() 函数和 help() 函数。这两个函数的一般使用格式为：

```
dir(对象名|模块名|函数名|...)
help(对象名|模块名|函数名|...)
```

读者可以输入以下代码，并观察 Python 给出的帮助信息。因帮助信息的内容很多，会占据大量篇幅，故此处不列出 Python 给出的帮助信息。

```
>>> import time
>>> dir(time)      # 以列表形式罗列 time 模块中的常量、私有函数、函数、类等信息
>>> help(time)     # 获取到 time 模块的整体帮助信息
>>> help(time.__eq__)
>>> help(time.sleep) # 获取 time 模块中 sleep 函数的帮助信息
>>> help(max)      # 获取内置函数 max 的帮助信息
```

2. 联机帮助环境

Python 还提供了联机帮助环境，以下是使用联机帮助的方法：

```
>>> help()          (必须是无参的 help()函数)
.....             (此处省略 Python 系统给出的提示)
help>              (出现该提示符，即表明已处在了联机帮助环境中)
help>math           (直接输入待查询的 math 模块)
Help on built-in module math: (本行开始是 math 模块的整体帮助信息)
.....             (此处省略 Python 系统给出的大量帮助信息)
help>quit           (退出联机帮助环境)
```

联机帮助环境可以直接查阅所有本机已安装模块的帮助信息，不需要事先导入模块。

2.7 习题

1. 下列标识符中，哪些是 Python 语言中有效的标识符名称。

John	\$123	_name	3D64
ab_c	2abc	char	a#3

2. Python 的数据类型分有哪两大类？它们各自又包含了哪些数据类型？

3. 请解释“Python 变量是没有数据类型的”的这种说法。

4. 在以下数据中，找出不是 Python 合法数据表示的项。并分别指出合法数据的数据类型。

100	9.123	true	'False'
-3 + 4i	e5	3.14E-3	0xFF
0b123	'I'm 9.\n'	""He said, "...."eg. ""	"goo\"d"
"\x6D09"	'123'	'[1,2,3]'	"

5. 程序中需要计算 a^3 ，写出至少两种正确的计算表达式。

6. 给出下列的数学公式对应的 Python 表达式。

$$(1) x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

$$(2) \frac{e^2 + \ln 10}{\sqrt{x} + 3^x}$$

$$(3) 10^{-5} \cos \frac{1}{2}(\alpha + \beta) \tan \frac{1}{2}(|\alpha - \beta|)$$

7. 如何判定两个计算结果为实数的表达式是否相等？

8. 写出实现以下功能的 Python 表达式。

(1) 将整数 k 转换为实数。

(2) 分别取出实数 m 中的整数部分和小数部分。

(3) 求正整数 m 中的每位上的数字，比如 123 的每位数字是 1、2、3。

9. 简述 Python 的标识符命名规则。

10. 求以下表达式的值：

(1) 设 $a=7$ 、 $x=2.1$ 、 $y=4.5$ ，求表达式 $x+a \% 3 * (x+y) \% 2 / 4$ 的值。

(2) 设 $a=2$ 、 $b=3$ 、 $x=3.6$ 、 $y=2.5$ ，求表达式 $(a+b) / 2 + x \% y$ 的值。

(3) 设 $x=2.5$ 、 $a=7$ 、 $y=4.7$ ，求表达式 $x+a \% 3 * (x+y) \% 2 // 4$ 的值。

第3章 顺序结构

Python 的一个程序是由若干条语句组成的，这些语句用来完成一定的操作任务。语句中包含各种成分，如常量、变量、表达式、函数等。

程序中语句的基本结构包括顺序结构、选择结构和循环结构。语句按照在程序中出现的先后次序依次执行的结构为顺序结构。根据条件选择执行不同语句的结构为选择结构。根据条件重复执行相关语句的结构为循环结构。

3.1 Python 程序的编码规范

3.1.1 一个 Python 程序

下面是一个 Python 程序，用于展示 Python 的编码的基本规范。程序中涉及的语法元素会在后面各章中陆续学习，本节不一一介绍。

【例 3-1】一个 Python 程序！

```
#1. """多行注释的第一行
#2.     多行注释的第二行
#3.     多行注释的第三行"""
#4. def exam(h):
#5.     x = eval(input("输入第一个数值: ")) # 输入字符串，并用 eval 函数转换
#6.     y = eval(input("输入第二个数值: "))
#7.     s = 0;    a = 1;    b = 2           # 变量 s、a、b 赋值为 0、1、2
#8.     # if 语句根据 x 值选择执行不同的语句块
#9.     if x > 0:                           # 当 x 大于 0 时，执行#10-#14 行代码
#10.         while h >= 0:                   # while 循环语句反复执行#10、#11 行代码
#11.             s = h + x
#12.             h = h - 1
#13.             y += h
#14.             print(y)
#15.     else:                             # 当 x 不大于 0 时，执行#16 行代码
#16.         print(x+y-h)
#17.     return h                          # 函数结束，并返回变量 h 的值
#18.
#19. print('本例演示缩进，本行为程序运行的首行语句')
#20. a = exam(5)                          # 调用自定义函数 exam
#21. print(a)                             # 输出变量 a 的值
```

运行程序，并且第一个数据输入 4.6，第二个数据输入 3.2，得到输出结果如下：

本例演示缩进，本行为程序运行的首行语句

输入第一个数值：4.6↵

输入第二个数值：3.2↵

2.2

-1

上面输出结果中的第二行是 input 函数执行时的提示，带下画线部分的内容 4.6↵ 是运行时敲击键盘输入的内容，行末的符号↵表示的是输入结束时所按的回车键。本书后面的写法与此相同。

例 3-1 的代码本身没有太多的实际用意，只是为了说明 Python 程序的编码基本规范。Python 的程序书写有严格的语法和文法要求，用户录入时有一点点的违规，例如大小写写错、少了一个字符或符号、符号录入成全角字符等，Python 的解释器都会报错。

3.1.2 注释

注释是对代码的说明信息，注释的存在与否不影响程序的功能，但建议在代码中增加适量的注释，以提高程序的可读性。注释有以下两种方式：

- 行末注释：在语句的行末增加#，并在其后添加注释的内容。
- 块注释：以三个单引号（'''）或三个双引号（"""）开头和结尾。

例 3-1 中的#1-#3 是一个块注释，#5、#7、#9、#10、#15、#17、#20、#21 是行末注释，在已有语句的末端添加注释，#8 的也是行末注释，但是并未包含有效语句，这也是#注释合法的用法。

在 IDLE 窗口中按快捷键 Alt+3 可以将当前行和所选代码块变为行末注释，按快捷键 Alt+4 可以取消行末注释。

3.1.3 缩进

在 Python 中，用不同的缩进表明代码块之间不同的层次关系，如图 3-1 所示。缩进默认采用四个空格，也可以由用户自己指定，但需保持同层代码块前的空格数量相同。缩进也可以使用 Tab 键，但不能是空格和 Tab 键混用。

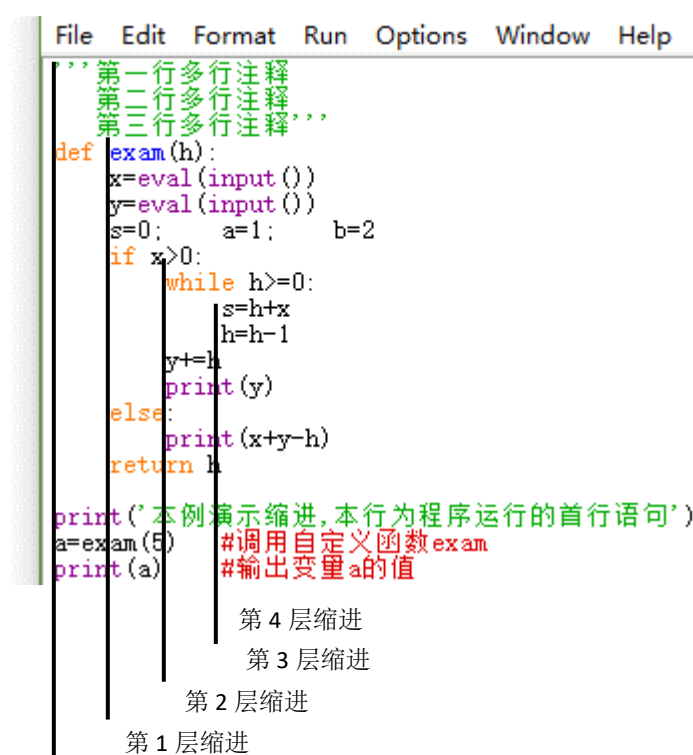


图 3-1 缩进与代码块间的关系

3.1.4 语句续行与分隔

Python 程序中的语句通常是一行写一条语句。

也可以将多个语句写在一行上，此时需要在多个语句之间加上分号（;），但行末无需添加分号。例 3-1 中的#7 就是一行中包含了三个赋值语句。

当 Python 的一条语句太长时，也可以将一条语句分成多行来写，只需在该语句的每行后使用反斜杠\，但最后一行末无需添加反斜杠。

若在语句中的小括号()、中括号[]、花括号{}的内部分行，则不需要使用换行符，三引号之间的内容也可以不加换行符，直接分多行书写。例如：

```
if a==1 and b==2 and \
    c==3 and d==4 and e==5:    # 此处可添加注释，上一行的\后不能添加行末注释
    print('OK')
# 以下分多行写的目的，是为了让字典的内容看起来更清楚了
# 因是在花括号的内部分行，可以不使用续行符
PCounts={'bcm':0,
        'bcf':0,
        'dcm':0,
        'dcf':0,
        'mnm':0,
        'mnf':0,
        'family':0,
        'famcount':0,
        'Normal':0,
        'Error':0}
```

3.2 赋值语句

3.2.1 赋值语句

1. 赋值的基本格式

赋值是建立变量和数据、对象、函数等之间的联系。赋值后就能通过变量使用数据、对象和函数了。赋值的基本格式：

```
var = obj
```

var 是变量，等号的右侧可以是常量、变量、表达式、对象和函数等。例如：

```
>>> a = 1
>>> b = 2
>>> c = 3
```

上述三行语句，对变量 a、b、c 分别赋值为 1、2、3。

2. 复合赋值

复合赋值是指将其他运算与赋值结合在一起。复合赋值包括+=、-=、*=、\=、\\=、%=、**=、<<=、>>=、&=、|=、^=。例如：

```
>>> x = 5
>>> x += 1          # 等价于 x = x + 1
>>> y = -1
>>> y *= x + 0.5    # 等价于 y = y * (x + 0.5)
>>> x, y
(6, -6.5)
```

注意：进行复合赋值运算时，先计算等号右侧的表达式值，再与左侧的变量进行运算，最后结果存回左侧的变量中。

3. 多变量赋值

- 链式赋值

格式：

```
var1 = var2 = var3... = 表达式
```

链式赋值用于将同一个值赋给多个变量。例如：

```
>>> a = b = c = 100
>>> a, b, c
(100, 100, 100)
```

- 同步赋值

格式：

```
var1[, var2[, var3...]] = 表达式 1[, 表达式 2[, 表达式 3...]]
```

注意等号左右两侧的变量数和表达式数要一致，然后，按位置将表达式值分别赋值给变量。例如：

```
>>> a, b, c = -2.3, 12, 'ab'
>>> a
-2.3
>>> b
12
>>> c
'ab'
```

同步赋值时，先计算出所有表达式的值后，再赋值给左侧的变量。

另外，Python 的同步赋值可以不借助第三个变量实现两个值的交换操作。例如，下面的两种方法都可以实现 a、b 两个数的内容交换。

方法一：

```
>>> a = 10
>>> b = 9.123
>>> t = a
>>> a = b
>>> b = t
>>> a
9.123
>>> b
10
```

方法二：

```
>>> a = 10
>>> b = 9.123
>>> a, b = b, a
>>> a
9.123
>>> b
10
```

方法一是借助了第三个变量 t 的实现方法，方法二则是采用同步赋值的方法，直接实现了两个数的交换。

3.2.2 变量的共享引用*

用赋值操作“=”将一个变量赋值给另一个变量时，要注意两个变量的值的变化和它们所引用空间的共享现象。本节讨论共享引用中的情况，不是为了把问题复杂化，而是在Python中改变变量值时的情况本身就是复杂的。编程者应该了解当前程序中采用的语句会引起变量值发生什么改变。切记：不能想当然的认为是某种结果，建议在使用某些语句前进行一定测试后，再写入到程序中。只有编程者清楚改变值会带来什么结果，程序才会有正确的运行结果。

(1) 数值类型和字符串的共享引用

当变量所赋的值为不可变数据类型，将该变量的值赋给别的变量时，多个变量都会指向同一个数据。下面是各种数值类型和字符串等不可变数据类型的共享引用示例。

整数 int 的共享引用示例：

```
>>> x = 1
>>> y = x
>>> x, y
(1, 1)
>>> id(x) == id(y)
True
>>> x is y
True
>>> x = 90
>>> x, y
(90, 1)
>>> id(x) == id(y)
False
```

实数 float 的共享引用示例：

```
>>> x = 3.1238
>>> y = x
>>> x, y
(3.1238, 3.1238)
>>> id(x) == id(y)
True
>>> x is y
True
>>> x = 5.678
>>> x, y
(5.678, 3.1238)
>>> id(x) == id(y)
False
```

复数 complex 的共享引用示例：

```
>>> x = 3+4j
>>> y = x
>>> x, y
((3+4j), (3+4j))
```

```

>>> id(x) == id(y)
True
>>> x is y
True
>>> x = -2-9j
>>> x, y
((-2-9j), (3+4j))
>>> id(x) == id(y)
False

```

字符串 `str` 的共享引用示例：

```

>>> x = 'Hi!'
>>> y = x
>>> x, y
('Hi!', 'Hi!')
>>> id(x) == id(y)
True
>>> x is y
True
>>> x = x + 'Bye!'
>>> x, y
('Hi!Bye!', 'Hi!')
>>> id(x) == id(y)
False

```

(2) 元组 `tuple` 的共享引用

元组被称为只读列表，即数据可以被查询，但不能被修改。当各个元组值是不可变数据类型时，其共享引用的方式与数值数据类型情况相同。元组中的元素是不可变数据的示例：

```

>>> x = (1, 2, 3)
>>> y = x
>>> x, y
((1, 2, 3), (1, 2, 3))
>>> x == y
True
>>> id(x) == id(y)
True
>>> x is y
True
>>> x = (6, 7, 8)
>>> x, y
((6, 7, 8), (1, 2, 3))
>>> x == y
False
>>> id(x) == id(y)
False
>>> x[1] = 100

```

```
Traceback (most recent call last):
  File "<pyshell#22>", line 1, in <module>
    x[1] = 100
TypeError: 'tuple' object does not support item assignment
```

若在元组中的元素是可变数据时，则该元组元素的数据值是可变的，但变量指向的元组空间没有变化。这是因为 Python 在元组的存储空间中存放的是数据的引用而不是数据值本身。元组中的元素有可变数据的示例：

```
>>> x = (1, 3, [10, 11])
>>> y = x
>>> x, y
((1, 3, [10, 11]), (1, 3, [10, 11]))
>>> x == y
True
>>> id(x)
1501033441536
>>> id(x) == id(y)
True
>>> x is y
True
>>> a[0] = 100
Traceback (most recent call last):
  File "<pyshell#18>", line 1, in <module>
    a[0] = 100
IndexError: list assignment index out of range
>>> x[2][0] = 99
>>> x, y
((1, 3, [99, 11]), (1, 3, [99, 11]))
>>> x == y
True
>>> id(x) == id(y)
True
>>> id(x)
1501033441536
>>> x is y
True
```

图 3-2 中左边的示意图展示了上面元组是不可变数据的示例代码中第一次赋值后 x 和 y 的变量引用情况，右边的示意图展示了上面元组是可变数据的示例代码中的变量引用情况。其中，灰色的列表值“10 的地址”后面变为了“99 的地址”，从而引发了显示元组时值的改变，但元组空间本身并没有改变，因此元组值的变化是间接引用引发的，元组仍然属于不可变数据类型。

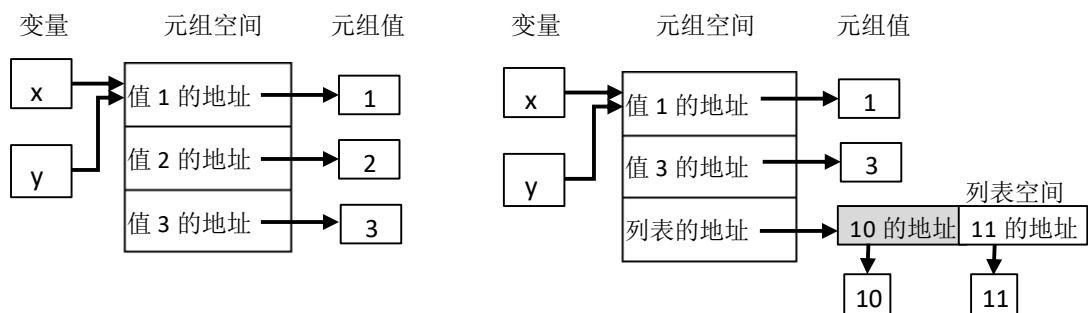


图 3-2 元组的共享引用

(3) 列表、字典、集合的共享引用

列表、字典、集合的元素是不可变数据时，改变某个元素值时，数据存储空间的位置不变，数据值发生改变。

列表的共享引用示例：

```
>>> x = [1, 2, 3]
>>> y = x
>>> x, y
([1, 2, 3], [1, 2, 3])
>>> id(x), id(y)
(2349538144328, 2349538144328)
>>> x[1] = 100
>>> x, y
([1, 100, 3], [1, 100, 3])
>>> id(x), id(y)
(2349538144328, 2349538144328)
>>> x is y
True
```

字典的共享引用示例：

```
>>> i = {'F':1, 'H':2, 'Y':3}
>>> j = i
>>> i
{'Y': 3, 'H': 2, 'F': 1}
>>> j
{'Y': 3, 'H': 2, 'F': 1}
>>> id(i), id(j)
(2349538516360, 2349538516360)
>>> i['F'] = 99
>>> i
{'Y': 3, 'H': 2, 'F': 99}
>>> j
{'Y': 3, 'H': 2, 'F': 99}
>>> id(i), id(j)
(2349538516360, 2349538516360)
>>> i is j
```

True

集合的共享引用示例:

```
>>> m = {3, 4, 5}
>>> n = m
>>> m, n
({3, 4, 5}, {3, 4, 5})
>>> id(m), id(n)
(2349538945064, 2349538945064)
>>> m.add(99)
>>> m, n
({99, 3, 4, 5}, {99, 3, 4, 5})
>>> id(m), id(n)
(2349538945064, 2349538945064)
>>> m is n
True
```

列表、字典、集合的元素是可变数据时，进行赋值操作或调用相关数据类型的方法改变某个可变数据元素值时，值的变化就会各不相同。例如：

```
>>> x = [1, 2, [9, 10]]
>>> y = x
>>> id(x)
2349539132360
>>> id(y)
2349539132360
>>> x=x+[4] #发生了浅复制
>>> x
[1, 2, [9, 10], 4]
>>> y
[1, 2, [9, 10]]
>>> id(x)
2349539131656
>>> id(y)
2349539132360
>>> x[0] = 100 # 独立引用
>>> x
[100, 2, [9, 10], 4]
>>> y
[1, 2, [9, 10]]
>>> y[2][0] = -1 # 共享引用
>>> x
[100, 2, [-1, 10], 4]
>>> y
[1, 2, [-1, 10]]
>>> id(x)
2349539131656
```

```
>>> id(y)
2349539132360
```

以上代码段发生了浅复制，图 3-3 所示的是语句 $x = x + [4]$ 执行前的引用情况，当执行 $x = x + [4]$ 时，Python 分配一个新的存储空间用于存储新的列表，并将原来 x 列表中的数据地址复制到新空间中，语句 $x[0] = 100$ 改变了新空间的引用值。图中的“间接列表空间”是由 x 和 y 共享的，语句 $y[2][0] = -1$ 改变了“间接列表空间”中的引用值，因而在显示 x 时，其值发生了改变。

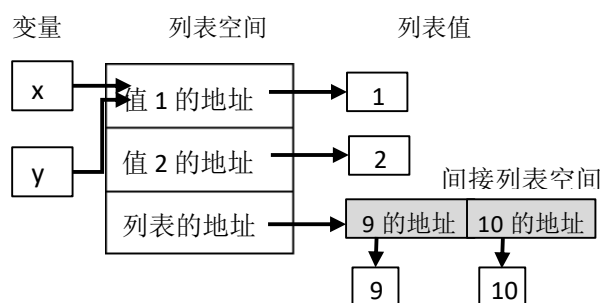


图 3-3 语句 $x = x + [4]$ 之前的内存状态图

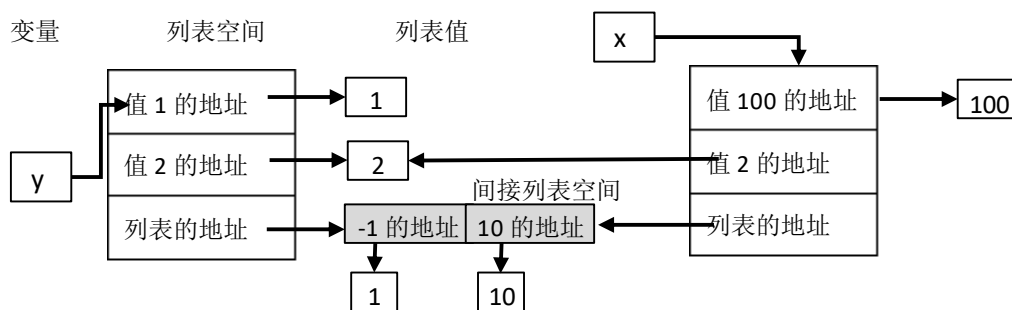


图 3-4 语句 $x = x + [4]$ 的浅复制后的内存状态图

下述代码，也发生了浅复制：

```
>>> x = [[1, 2, 3]] * 3
[[1, 2, 3], [1, 2, 3], [1, 2, 3]]
>>> x[0][0] = 10
>>> x
[[10, 2, 3], [10, 2, 3], [10, 2, 3]]
```

执行语句 $x = [[1, 2, 3]] * 3$ 后，新列表的三个数据元素都指引到同一个间接列表空间，语句 $x[0][0] = 10$ 貌似只改变了一个间接值，但结果是引发了连锁反应。

与浅复制相对应的是深复制，例如有如下代码：

```
>>> import copy
>>> names = ["小明", "小红", "小黑", ["粉色"], "小黄", "小白"]
>>> deep_names = copy.deepcopy(names)
>>> names[3][0] = "Pink"
>>> names
['小明', '小红', '小黑', ['Pink'], '小黄', '小白']
>>> deep_names
['小明', '小红', '小黑', ['粉色'], '小黄', '小白']
```

上述代码中调用 `copy` 模块中的 `deepcopy` 函数将发生深复制,使两个变量的值各自独立,其中的细节,请读者另行查阅资料或自行研究。

3.2.3 对象的垃圾回收机制*

Python 中万物皆对象,在前面已介绍的内容中,每个数据都是一个对象。后面在第 11 章还将专门介绍关于对象的知识。在赋值操作中,Python 会建立变量和对象之间的引用关系(如图 2-1 所示)。程序中需要使用数据或对象时,就会在内存中开辟一块内存空间存放该数据或对象。

(1) Python 对象的存储问题

- 对于整数和短小的字符串等,Python 会执行缓存机制,即将这些对象进行缓存,不会为相同的对象分配多个内存空间。
- 容器对象(如列表、元组、字典等)存储的其他对象,仅仅是其他对象的引用,即记录了对象的地址,并不是这些对象本身。例如:

```
>>> import sys
>>> sys.getrefcount(3)    # 3 在 Python 内部已经被多次引用,故引用计数大于 2。
184
>>> h = 3
>>> x = [3, 3, 3]
>>> sys.getrefcount(3)
188
>>> id(h), id(x[0]), id(x[1]), id(x[2])
(1900216848, 1900216848, 1900216848, 1900216848)
>>> sys.getrefcount('OK')
21
>>> sys.getrefcount(123)
4
>>> sys.getrefcount(3.1)
2
```

上述代码中的 3、'OK'、123 属于整数和短小字符,Python 执行缓存机制。实数 3.1 被 `getrefcount()` 查看引用计数时,3.1 作为参数传递给了 `getrefcount()`,参数实际上创建了一个临时的引用。因此,`getrefcount()` 所得到的结果,会比期望的多 1。

(2) 关于引用计数器

在 Python 中,每个对象都有指向该对象的引用总数,即引用计数。每增加一个引用,引用计数加 1,每减少一个引用,引用计数减 1。

会增加引用计数的几种情况有:

- 对象被创建,计数增加 1;
- 对象被引用,计数增加 1;
- 对象被当作参数传入函数,计数增加 2,因为调用函数中有两个属性在引用该对象;
- 对象存储到容器对象中,计数增加 1。

会减少引用计数的几种情况有:

- 对象的别名被销毁,计数减少 1;
- 对象的别名被赋予其他对象,计数减少 1;
- 对象离开自己的作用域,计数减少 1;

- 对象从容器对象中删除，或者容器对象被销毁，计数减少 1。

使用 `sys` 模块中的 `getrefcount()` 函数可以查看对象的引用计数。例如：

```
#1. import sys
#2. class Person(object):
#3.     pass
#4. p = Person()
#5. p1 = p
#6. print(sys.getrefcount(p))
#7. p2 = p1
#8. print(sys.getrefcount(p))
#9. p3 = p2
#10. print(sys.getrefcount(p))
#11. del p1
#12. print(sys.getrefcount(p))
```

利用引用计数器方法，在检测到对象引用个数为 0 时，对普通的对象进行释放内存的机制处理（该机制如何处理，本书不展开讨论）。

（3）垃圾回收

当 Python 中的对象越来越多时，它们将占据越来越大的内存。Python 会在适当的时候，启动垃圾回收（`garbage collection`），将没用的对象清除。

垃圾回收时，Python 不能进行其他的任务。频繁的垃圾回收将大大降低 Python 的工作效率。如果内存中的对象不多，就没有必要总启动垃圾回收。所以，Python 只会在特定条件下，自动启动垃圾回收。

3.3 数据的输入/输出

3.3.1 标准输入/输出

被程序处理的数据可以从键盘输入，也可以从文件读入。程序的处理结果则可以显示在屏幕上，或存入文件。所谓的标准输入/输出是指从键盘输入和在屏幕显示，又叫控制台输入/输出

1. 标准输入

Python 中的 `input` 函数用于实现标准输入，其格式为：

```
input(prompt=None)
```

参数 `prompt` 是输入时的提示文字，属于可选项。`input` 函数将接受标准输入设备键盘输入的一行信息，并返回一个字符串。返回的字符串是去除行末回车符后的输入内容。例如：

```
>>> input('Please input your name:')
Please input your name:Sam↵
'Sam'
```

上述示例中，第 2 行前面的文字是 `input` 函数执行时的提示，`Sam` 是运行时敲击键盘输入的内容。第 3 行是函数的返回值，两个单引号表示返回值是一个字符串。

如果输入的内容是整数或浮点数，则需要使用 `int()` 或 `float()` 函数进行转换。例如：

```
#1. >>> input('Please input your age:')
#1. Please input your age:19↵
#2. '19'
#3. >>> int(input('Please input your age:'))
```



```
#4. Please input your age:19✓
#5. 19
#6. >>> float(input('Please input your score:'))
#7. Please input your score:75.5✓
#8. 75.5
```

上述示例中#6 和#9 行显示的返回值中无单引号，即其返回值分别为整数类型和浮点数类型。

2. 多数据同时输入并转换

单独使用 `int` 或 `float` 函数只能输入一个数据并进行数据类型转换，若想要一次输入多个数据并转换成整型或浮点数，则可以用以下几种方法。

- 利用 `eval()` 函数实现多数据同时输入

`eval` 函数可以计算参数字符串中的表达式或通过 `compile()` 执行一个代码对象，其格式为：

```
eval(source, globals=None, locals=None)
```

参数 `source` 是一串待计算的表达式或待执行的代码。例如：

```
>>> eval('help(eval)')
Help on built-in function eval in module builtins:

eval(source, globals=None, locals=None, /)
    Evaluate the given source in the context of globals and locals.

    The source may be a string representing a Python expression
    or a code object as returned by compile().
    The globals must be a dictionary and locals can be any mapping,
    defaulting to the current globals and locals.
    If only globals is given, locals defaults to it.
```

通过函数 `eval` 执行字符串 `'help(eval)'` 中的 `help(eval)` 函数，`help` 函数返回 `eval` 函数的帮助信息。又如：

```
>>> a = 100
>>> b = eval('a / 2.0')
>>> b
50.0
```

上述代码中，`b=eval('a / 2.0')` 等价于 `b = a / 2.0`，所以 `b` 的值变为 50.0。

再比如，可用如下代码输入多个数据到多个变量：

```
#1. >>> a, b, c = eval(input('a,b,c='))
#2. a,b,c=1,2,3✓
#3. >>> a
#4. 1
#5. >>> b
#6. 2
#7. >>> c
#8. 3
```

上述代码中，#2 行中输入的 1,2,3 使得#1 的代码等价于 `a, b, c = 1, 2, 3`

- 利用字符串的切片和 `map()` 函数实现多数据同时输入

字符串的切片格式:

```
S.split(sep=None, maxsplit=-1) -> list of strings
```

S 是被处理的字符串, 参数 `sep` 是切片字符, `maxsplit` 是切片的次数。函数返回一个包含切片后子串的列表。省略 `sep`, 则切片字符是空白字符 (包括空格、Tab 和回车\n), 省略 `maxsplit`, 则不限制切片次数, 即遇到参数 `sep` 指定的字符都切片。例如:

```
>>> '1 2 3'.split(' ',1)          # '1 2 3'的 1、2、3 之间有个空格, 共切片 1 次
['1', '2 3']
>>> '1 2\t3\n4 5'.split()         # 字符串的 1、2 之间和 4、5 之间有个空格
['1', '2', '3', '4', '5']
>>> '1,2,3,4,5'.split(',')
['1', '2', '3', '4', '5']
>>> input('Please input 5 number:').split()
Please input 5 number:12 3 4 5 ✓
['1', '2', '3', '4', '5']
```

从上述示例代码中, 可以观察到通过字符串切片函数可以将同时输入的整型或浮点数分离出来, 但是分离出来的内容还是字符串, 而不直接是整数或浮点数。`int()`或 `float()`函数只能转换一个数, 如果要将多个字符串转换成同种数据类型结果则可以使用 `map()`函数。

`map()`可以让单参数的函数作用到序列或可迭代对象上, 返回一系列的处理结果。其格式为:

```
map(func, *iterables)
```

调用 `map` 函数时, `func` 参数应设为单参数函数的函数名, 参数 `*iterables` 为序列或可迭代对象, 作用是将每个元素传递给单参数处理, 得到各项函数值, 并以 `map` 对象返回。例如:

```
>>> x = input('Please input 5 number:')
Please input 5 number:1;2;3;4;5 ✓
>>> y = x.split(';')
>>> y
['1', '2', '3', '4', '5']
>>> map(int, y)
<map object at 0x000001DE8A273390>
>>> z=map(int, y)
>>> z          # 返回的 map 对象是可迭代对象, 但不能直接显示其内部各元素值
<map object at 0x000001DE8A273438>
>>> w = list(z)
>>> w
[1, 2, 3, 4, 5]
>>> w = list(map(int, input('Please input 5 number:').split(';')))
Please input 5 number:1;2;3;4;5 ✓
>>> w
[1, 2, 3, 4, 5]
```

3. 标准输出

程序执行中产生的处理结果, 需要以一定方式展示出来, 其中最常用的方法是显示在屏幕上。Python 中的标准输出函数是 `print()`, 其格式为:

```
print(value,..., sep=' ', end='\n', file=sys.stdout, flush=False)
```

其作用：显示参数 `value` 到输出流或标准输出设备（即屏幕）`sys.stdout`。参数 `file` 用于指定输出流，缺省 `file` 则输出到屏幕。参数 `sep` 指定多输出项之间的间隔字符，缺省 `sep` 则以空格间隔。参数 `end` 指定显示最后一项 `value` 后显示的字符，缺省 `end` 则显示 `\n`，即默认情况，显示数据后会换行。例如：

```
>>> print(1, 'OK', 98.12, [1, 2, 3], None)
1 OK 98.12 [1, 2, 3] None
>>> print(1, 'OK', 98.12, [1, 2, 3], None, sep='!'); print('----')
1!OK!98.12![1, 2, 3]!None
----
>>> print(1, 'OK', 98.12, [1, 2, 3], None, sep='!', end=' '); print('----')
1!OK!98.12![1, 2, 3]!None ----
```

3.3.2 格式化输出

很多应用中，对输出内容是有格式要求的。比如，很多实验数据需要保留指定位数的小数，而使用 `print` 函数直接输出时，小数位数是 Python 内部自动控制的。例如：

```
>>> a = 19 / 7
>>> a
2.7142857142857144
```

Python 中，可以用以下方法控制输出内容的格式：

- 利用字符串格式化运算符`%`；
- 利用内置函数 `format()`；
- 利用字符串的 `format()` 方法。

1. 字符串格式化运算符`%`

这是 Python 的早期版本提供的一种输出格式化方法。字符串格式化运算符`%`的使用格式：

格式字符串 `%(数据项 1,[数据项 2,[数据项 3,...]])`

格式字符串可以有普通字符和格式字符组成，普通字符按原样输出，而一组格式字符与一个数据项对应，由以下内容组成：

`% [-] [+][0][m][.n]` 数据类型说明符

格式字符串由`%`开始，数据类型说明符则根据数据项的数据类型来指定，如表 3-1 所示。

表 3-1 格式字符串中不同符号的含义

格式符号	格式化结果
<code>%%</code>	字符百分号 <code>%</code> 。
<code>%c</code>	单个字符。
<code>%s</code>	字符串，等价于 <code>str()</code> 的返回值。
<code>%r</code>	字符串，等价于 <code>repr()</code> 的返回值。
<code>%d</code> 或 <code>%i</code>	十进制整数。
<code>%o</code>	八进制整数。
<code>%x</code> 或 <code>%X</code>	十六进制整数，其中的字符用小写或大写。
<code>%e</code> 或 <code>%E</code>	科学计数法表示的浮点数，用 <code>e</code> 或 <code>E</code> 表示。
<code>%f</code> 或 <code>%F</code>	非科学计数法表示的浮点数。
<code>%g</code> 或 <code>%G</code>	浮点数，系统自动根据值的大小采用 <code>%e</code> 、 <code>%E</code> 、 <code>%f</code> 或 <code>%F</code> 。
<code>-</code>	左对齐输出。

格式符号	格式化结果
+	对正数加正号。
0	空位用 0 填充。
m	m 是数字，指定最小宽度。
.n	n 是数字，指定精度，采用 %e、%E、%f、%F、%g 或 %G 时含义不同。

下面的示例展示字符串格式化运算符 % 的用法。

```
>>> print('%%\t%c\t%s\t%r' % ('A', 'abc', 'abc'))
%  A   abc  'abc'
```

这里的 %% 将输出一个 %，后面紧跟着一个 \t，表示跳过一个制表位，接着 %c 对应第一个输出项 'A'，以单个字符形式显示为 A，后面又是一个 \t，继续跳过一个制表位，接着 %s 对应的是第二个输出项 'abc'，以 %s 格式化字符串时不显示字符串的定界符，然后又是跳过一个制表位后，%r 对应了第三个输出项 'abc'，用 %r 格式化字符串时会显示字符串的定界符。

```
>>> print('%d, %i, %o, %x, %X' % (299, 299, 299, 299, 299))
299, 299, 453, 12b, 12B
```

这个示例中 5 个输出项分别对应了 299 的十进制、十进制、八进制、十六进制的小写形式和十六进制的大写形式。

```
>>> print("Name: %s Age: %d Height: %f" % ("Aviad", 25, 1.83))
Name: Aviad Age: 25 Height: 1.830000
```

上面 3 个输出项都是根据各自的数据类型采用了 Python 默认的输出格式。

```
>>> print("Name: %10s Age: %8d Height: %8.2f" % ("Aviad", 25, 1.83))
Name:      Aviad Age:      25 Height:      1.83
```

这里的输出项都指定了宽度，当指定的宽度大于数据长度时，将在前面补空格填满指定宽度。输出 1.83 时，还规定了小数点后保留 2 位，即 %8.2f 表示宽度为 8 位，精度为 2 位。

```
>>> print("Name: %-10s Age: %-8d Height: %-8.2f" % ("Aviad", 25, 1.83))
Name: Aviad      Age: 25      Height: 1.83
```

格式字符串中的 - 代表左对齐，当指定宽度时，补的空格是在数据的右边。

```
>>> print("Name: %010s Age: %08d Height: %08.2f" % ("Aviad", 25, 1.83))
Name:      Aviad Age: 00000025 Height: 00001.83
```

格式字符串中含有 0，表示填充字符是 0，而不是空格。

```
>>> print("Name: %010s Age: %0+8d Height: %+08.2f" % ("Aviad", 25, 1.83))
Name:      Aviad Age: +0000025 Height: +0001.83
```

格式字符串中含有 +，表示正数前面要有 +。

```
>>> print("Name: %(name)010s Age: %(age)0+8d Height: %(height)+08.2f" % \
        {'name': 'Aviad', 'height': 1.83, 'age': 25}) # 输出字典数据时的用法
Name:      Aviad Age: +0000025 Height: +0001.83
```

格式字符串中出现在括号内的是字典的键，输出时会在括号处替换为与键对应的值。

```
>>> '%g' % 12345.456789901234 # 自动根据数据大小选择小数形式或指数形式
'12345.5'
>>> '%f' % 12345.456789901234 # 小数形式
'12345.456790'
>>> '%e' % 12345.456789901234 # 指数形式
'1.234546e+04'
>>> '%14.5g' % 12345.456789901234 # 自动形式中的精度指有效数字数
'12345'
```

```
>>> '%14.5f' % 12345.456789901234 # 小数形式中的精度指小数点后的位数
' 12345.45679'
>>> '%14.5e' % 12345.456789901234 # 指数形式中的精度指前面数字的小数位数
' 1.23455e+04'
>>> '%-+0*.*f' % (16, 2, 100.93) # 在运算符%后指定最小宽度和精度,
'+100.93'
```

上面最后一个示例比较特殊, 里面出现了*.*, 这两个*分别对应的是括号中的前两项 16 和 2, 相当于是 16.2, 所以虽然括号中有三个数字, 但输出项只有最后的那个 100.93。

2. format()函数

format()函数用于将单项数据格式化, 其格式为:

```
format(输出项[, 格式字符串])
```

当省略第二个参数时, format 函数等价于 str(), 即将输出项转换为字符串。格式字符串中的基本格式控制符如下:

- d、b、o、x、X 分别用十进制、二进制、八进制和十六进制输出整数。例如:

```
>>> print(format(95, 'X'), format(95, 'o'), format(95, 'b'))
5F 137 1011111
```

- f 或 F、e 或 E、g 或 G 分别用小数形式、科学计数和自动判定来输出浮点数。例如:

```
>>> print(format(162.28193, 'e'), format(162.28193, 'g'), format(162.28193, 'f'))
1.622819e+02 162.282 162.281930
```

- c 输出字符, 字符的 ASCII 码由参数输出项指定。
- %输出百分数, 数值由输出项指定。
- ,输出浮点数时, 带千分位符, 例如:

```
>>> print(format(31416.123, 'f'))
31,416.123000
```

注意小数点后 123 与 000 之间是没有千分位符的, 因为这里的 000 不是小数点后的有效位数, 只是将空白区填充满 0, 真正有效的小数位数只有 3 位。

- 用形如 m.n 的格式来控制输出宽度和精度, m 和 n 都是数字。
- 输出整数或浮点数时, 可以使用+表示正数带正号。
- 在指定输出宽度时, 在输出宽度前可以使用 0、<、>、^表示用 0 填充空位 (默认用空格填充空位)、左对齐、右对齐和居中对齐。

以上格式控制符根据数据类型的不同, 可以部分同时使用。例如:

```
>>> print(format(2.11, '10'), format(2.11, '010'), format(2.11, '+10'))
2.11 0000002.11 +2.11
```

这个示例中输出了三次 2.11, 它们的宽度都是 10, 第一个是左边补空格, 第二个是左边补 0, 第三个是 2.11 前要出现+。

```
>>> print(format('aaa', '<10'), '|', format('aaa', '^10'), '|', format('aaa', '>10'), '|')
aaa | aaa | aaa |
```

这个示例是左对齐、居中和右对齐的示例。

```
>>> print(format(3.1416, '8.3f'), '|', format(3.1416, '08.3f'), '|', format(3.1416, '+08.3f'))
3.142 | 0003.142 | +003.142
>>> print(format(3.1416, '<8.3f'), '|', format(3.1416, '<08.3f'), '|', format(3.1416, '<+08.3f'))
3.142 | 3.142000 | +3.14200
```

上面两个示例都是浮点数的输出示例, 可以控制宽度和精度, 也可以控制对齐方式。

3. 字符串的 format() 方法

Python 中的字符串类型有一个 `format()` 方法，利用该方法可以格式化字符串。字符串 `format()` 方法的调用格式为：

格式字符串 `.format([键名 0=]输出项 0,[键名 1=]输出项 1,[键名 2=]输出项 2,...)`

格式字符串中可以包括普通字符和格式说明模板，可以有多个格式说明模板，普通字符原样输出。格式说明模板的格式为：

`{{输出项序号|键名[:格式说明符]}}`

其中，`{}` 是输出模板的定界符，输出项序号为 0、1、2、...，分别对应输出项 0、输出项 1、输出项 2...。格式说明模板中的键名与输出项前的键名匹配。省略输出项序号和键名时，多个格式说明模板与多个输出项按自然位置对应显示。除数字以外的格式说明符与内置函数 `format()` 中的格式说明符含义基本一致。例如：

```
>>> print('Name: {0} Age: {2} Height: {1}'.format("Aviad", 1.83, 25))
Name: Aviad Age: 25 Height: 1.83
```

这个示例中 `{}` 内的是序号，0 对应第一个输出项 "Aviad"，1 对应第二个输出项 1.83，2 对应第三个输出项 25。

```
>>> print('Name: {name} Age: {height} Height: {age}'.format(name="Aviad",
    height=1.83, age=25))
Name: Aviad Age: 1.83 Height: 25
```

这个示例中 `{}` 内的是字典的键，`name` 对应 "Aviad"，`height` 对应 1.83，`age` 对应 25。

```
>>> print('{0:010b}|{0:>10o}|{0:^10x}|{0:<10X}'.format(95))
0001011111|      137|      5f      |5F
>>> print('{0:018}|{0:>18}|{0:^18.2}|{0:<18.3}'.format(3.14159))
0000000000003.14159|      3.14159|      3.1      |3.14
```

这两个例子都是同时规定序号和格式的例子。在 “:” 前面的是序号，本例中都是 0，对应的是第一个输出项；在 “:” 后面的是格式控制。

```
>>> print('{:14}|{:>14}|{:<14}'.format('test', 'test', 'test'))
test      |      test|test
```

“:” 前面的序号可以省略，当省略序号时，自动根据格式说明模板的顺序，与输出项按照自然顺序相对应。

```
>>> print('{0:*>18}|{0:??^18}|{0:-<18}'.format('test'))
*****test|???????test???????|test-----
```

字符串类型的 `format()` 方法对于填充字符，若输出项是整数或浮点数，填充字符只能是 0 或空格，默认为空格，若输出项是字符串则可以指定任意填充字符。

3.4 顺序结构程序举例

1. 程序编写的一般方法

拿到一个问题，到最终编写出正确的程序来处理这个问题，一般需要经历以下步骤：（1）分析问题；（2）确定算法；（3）编写代码；（4）测试代码；（5）提交并发布代码。

大多数程序的基本功能框架如下：

- 获取原始数据
- 对数据进行处理
- 输出或存储处理结果

2. 顺序结构程序举例

程序的主结构是顺序结构，即程序总是按照语句出现的先后顺序依次执行的。以下是一些顺序结构的示例，通过它们可以对顺序结构、编程的一般步骤和程序基本功能框架有一定的了解。

【例 3-2】A 汽车从甲地开往乙地，以平均速度 45 公里/小时行驶，B 汽车从乙地开往甲地，以平均速度 53 公里/小时行驶，两辆车行驶了 2 小时 13 分钟后相遇。编写程序，输入 A、B 的速度和行驶的时间，求甲乙两地相距多少公里？

以下是本例按编程的一般步骤示范的解题过程。本书后续的其他示例一般只给出问题的解题思路和程序代码。

(1) 分析问题：这是个数学问题，找到基本的数学公式“距离=(A 速度+B 速度)*行驶时间”就能求解。

(2) 确定算法：算法流程如图 3-5 所示，按程序三部曲（输入、处理、输出）结合本问题的特点，确定算法的每一步。

(3) 编写代码：根据算法，将每个步骤落实为具体的代码。本题的程序代码如下：

```
#1. va,vb = eval(input('输入 A 车和 B 车的速度: '))
#2. h, m = eval(input('输入行驶时间: '))
#3. time = h + m / 60
#4. s = time * (va + vb)
#5. print('甲乙两地相距{}公里.'.format(s))
```

(4) 测试代码：在 IDLE 中，输入上述代码，并参考本书第 12 章中的介绍，先改正语法错误、运行时错误后，输入若干测试数据，观察每次运行结果的正确性。以下是某次测试的结果。

```
输入 A 车和 B 车的速度: 45,53✓
输入行驶时间: 2,13✓
甲乙两地相距 217.23333333333335 公里。
```

如果发现运行结果不正确则程序必然存在逻辑错误，需要逐行分析代码或采用 12.4 节的调试工具来发现错误点，并改正。若所有测试数据都有正确的运行结果，则可以进行最后一步提交并发布代码。

【例 3-3】解析几何中，求点(x,y)到直线 $Ax+By+C=0$ 的距离公式是 $\frac{|Ax+By+C|}{\sqrt{A^2+B^2}}$ ，编写程序输入点的坐标(x,y)值和直线方程的参数 A、B、C 的值，求点到直线的距离。

解题过程：

距离公式中涉及到求绝对值、求平方和求开平方，可以使用内置 `abs()` 函数求绝对值，平方就是一个数乘以自身，开平方可以用乘方运算`**`求解。

程序代码如下：

```
#1. x, y = eval(input('x,y:'))
#2. a, b, c = eval(input('方程系数 A,B,C:'))
#3. s = abs(a * x + b * y + c) / (a * a + b * b) ** 2
#4. print('点到直线的距离: {}'.format(s))
```

以下是某次测试结果：

```
x,y:0,1✓
方程系数 A,B,C:2,3,4✓
点到直线的距离: 0.04142011834319527
```

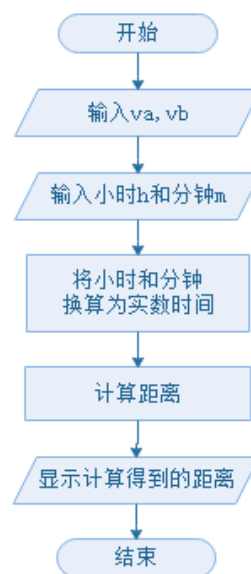


图 3-5 例 3-5 的流程图

【例 3-4】从键盘输入一个 3 位整数，计算该数中各位数字之和。例如，输入 392，各位数字之和是 $3+9+2=14$ 。

解题思路：

本题的难点是如何将整数中的各位数字提取出来，以下是三种提取数字的方法。

方法一：

```
#1. x = input('请输入一个三位数：')
#2. x = int(x)
#3. a = x // 100          # 获取百位数
#4. b = x // 10 % 10     # 获取十位数
#5. c = x % 10           # 获取个位数
#6. print(a+b+c)
```

方法二：

```
#1. x = input('请输入一个三位数：')
#2. x = int(x)
#3. a, b = divmod(x, 100)  # a 得到百位数，b 得到后两位数
#4. b, c = divmod(b, 10)   # b 得到十位数，c 得到个位数
#5. print(a+b+c)
```

方法三：

```
#1. x = input('请输入一个三位数：')
#2. a, b, c = map(int, x)   # 采用 map 函数依次得到百、十、个位数
#3. print(a+b+c)
```

这三种方法都可以得到同样的结果。下面是某次的测试结果：

```
请输入一个三位数： 392✓
14
```

【例 3-5】从键盘输入一个 3 位整数，将该整数转换为英文表达。例如，输入 392，输出 three hundred and ninetytwo。

有人设计了如下代码：

```
#1. x = input('请输入一个三位数：')
#2. a, b, c = map(int, x)
#3. eng1=["one", 'two', 'three', 'four', 'five', 'six', 'seven', 'eight', 'nine']
#4. eng2=["ten", 'twenty', 'thirty', 'forty', 'fifty', 'sixty', 'seventy', 'eighty', 'ninety']
#5. print('{} hundred and {}-{}'.format(eng1[a], eng2[b], eng1[c]))
```

上述代码的解题思路是：列表 eng1 中存放单个数字对应的英文单词，eng2 中存放 10、20……等整十数的英文单词，a、b、c 是分离出来的各位数字，用 a、b、c 做为列表下标去获取数字对应的英文单词。

运行程序，输入如下数据测试代码：

```
请输入一个三位数： 392✓
three hundred and ninety-two
```

以上数据测试正确。但是，实际上本题的解法是有缺陷的，当末两位是 11~19 或末尾为 0 时翻译结果是不正确的，要解决这个问题就需要用到选择结构。以下是错误的运行结果：

```
请输入一个三位数： 312✓
three hundred and ten-two
```

3.5 习题

1. 简述 Python 程序中注释的格式。
2. 简述 Python 程序中缩进的作用。
3. Python 的语句是如何将多个语句写在一行上的？Python 对将一个语句写到多行上有哪些规定？
4. Python 有哪几种赋值语句？简述各自的格式。
5. 简述 Python 是如何实现输入和输出的？
6. 求 x 的值：
 - (1) 已知 $x=3$ 、 $y=2$ ，执行表达式 $x*=y+8$ 后，x 的值。
 - (2) 已知 $x=10$ ，执行表达式 $x+=x$ 后，x 的值
 - (3) 执行表达式 $y,z,x=4,16,32$ 后，x 的值。
7. 运行以下程序时，输入 “1,2,3”，写出运行结果。


```
#1. i = input('Please input:')
#2. a, b, c = eval(i)
#3. m, n, r=i.split(',')
#4. x, y, z=map(int, i.split(','))
#5. print(a, b, c, m, n, r, x, y, z)
```
8. 运行以下程序时，输入 “123”，写出运行结果。


```
#1. x = input('Please input:')
#2. i, j, k = map(int, x)
#3. print(i, j, k)
```
9. 写出下列程序的运行结果。
 - (1)


```
#1. print('Hi,world', end='')
#2. print("I'm ...")
```
 - (2)


```
#1. a = 'Hi,world'
#2. b = "I'm ..."
#3. print(a, b, a, b, a, b, sep='\n')
#4. print(b)
#5. print('toot tied!')
```
 - (3)


```
#1. a = '甲'
#2. b = '乙'
#3. c = '丙'
#4. print('第一名: {}, 第二名: {}, 第三名: {}'.format(a, b, c))
#5. print('第一名: {2}, 第二名: {0}, 第三名: {1}'.format(a, b, c))
#6. print('第一名: {s2}, 第二名: {s1}, 第三名: {s3}'.format(s1=a, s2=b, s3=c))
#7. format(s1=a, s2=b, s3=c))
```
10. 请计算出下列语句中各个赋值运算符左边的变量的值。注意，并不是按顺序执行这些语句，假定在每条前都已安排下列语句：


```
#1. i = 3;
#2. j = 5;
#3. x = 4.3;
#4. y = 58.209;
```

- (1) `k = j * i;`
- (2) `k = j / i;`
- (3) `z = x / i;`
- (4) `k = x // i;`
- (5) `z = y / x;`
- (6) `k = y // x;`
- (7) `i = 3 + 2 * j;`
- (8) `k = j % i;`
- (9) `k = j % i * 4;`
- (10) `i += j;`
- (11) `j -= x;`
- (12) `i %= j`

11. 编写程序，实现从键盘输入学生的平时成绩、期中成绩、期末成绩，计算学生的学期总成绩。学生的学期总成绩=平时成绩*15%+期中成绩*25%+期末成绩*60%。
12. 编写程序，从键盘输入一个 3 位整数，计算该数的逆序数。例如，输入 392，输出 293。
13. 对于一元二次方程 $ax^2+bx+c=0$ ，输入其三个系数 a、b、c，输出方程的根。注：Python 有复数类型，且可以用 `sqrt()` 直接求得复数结果。
14. 有 4 个进水管（A、B、C、D）可以往水箱里注水，如果单开 A，则 a 小时注满；如果单开 B，则 b 小时可以注满；如果单开 C，则 c 小时注满；如果单开 D，则 d 小时可以注满。编写程序，输入 a、b、c、d 的值，计算 4 个水管同时注水，注满水箱需要多少小时？

第4章 选择结构

能自动根据不同情况选择执行不同的程序功能是对计算机程序的一个基本要求。这样的控制可以用选择结构来实现。Python 提供了条件运算符和选择控制语句两种选择结构。条件运算符是一种简单的选择结构,复杂的选择结构则要通过选择结构的控制语句 if 语句来实现。

4.1 条件表达式

在选择结构和循环结构中,程序的执行是根据条件表达式的运算结果来选择下一步如何执行的。Python 中条件表达式可以是单个常量、变量或合法的任意表达式。条件表达式中可以出现前面介绍过的所有运算符,本节还将介绍与条件判断相关度更高的关系运算符、逻辑运算符、测试运算符和一些测试函数。

在进行条件判断时,条件表达式的计算结果对应到布尔型 (bool) 的两个值 True 或者 False。一般来说, True 表示条件成立, False 代表条件不成立。当条件表达式的结果值不直接为 True 或 False 时, None、任何数值类型中的 0 (或 0.0、0j 等)、空字符串""、空元组()、空列表[]、空字典{}、空集合和结果为空的迭代对象等都等价于 False, 其他值则等价于 True。

4.1.1 关系运算

关系运算符用于比较两个操作数的大小关系。Python 提供了以下 6 种关系运算符:

<	小于
<=	小于等于
>=	大于等于
>	大于
==	等于
!=	不等于

关系运算符中,被比较的数一般为相同类型的可比较数据,不同数据类型间比较会出错,但整型和浮点型之间可以进行比较。例如:

```
>>> print('abc' == 'abcd')
False
>>> print(0 < 12.5)
True
>>> print(4 > 'abc')
Traceback (most recent call last):
  File "<pyshell#0>", line 1, in <module>
    4 > 'abc'
TypeError: unorderable types: int()>str()
>>> print(10 > -3+5j)
Traceback (most recent call last):
  File "<pyshell#3>", line 1, in <module>
    10 > -3+5j
TypeError: unorderable types: int()>complex()
>>> print([1, 2, 3] > [1, 2])
True
>>> print((1, 2, 3) < (2, 3, 4))
```

```
True
>>> print({1, 2, 3} >= {2, 3, 4})
False
```

Python 允许在一个关系表达式中比较多个值，但大小关系不具有传递性，仅当表达式中多个关系运算的计算结果都为 True 时，才显示 True 的结果。例如：

```
>>> a, b, c = 1, 2, 3
>>> print(a<b>c)           # 判断是否 a<b 并且 b>c
False
>>> print(a<b<c)           # 判断是否 a<b 并且 b<c
True
```

4. 1. 2逻辑运算

逻辑运算是多个布尔型 (bool) 值进行运算，一般用于表达多个条件之间的相互关系。Python 提供了以下 3 种关系运算符：

not	取反运算
and	与运算
or	或运算

Python 中参与逻辑运算的操作数可以为非布尔型 (bool) 数据，逻辑运算的结果也可以为非布尔型 (bool) 数据。逻辑运算符的运算规则和注意事项如下：

- 取反运算 not: 将 True 或与 True 等价的数据变为 False，False 或与 False 等价的数据则变为 True。例如：

```
>>> a = True
>>> not a
False
>>> print(not (9%3), not (9//3))
True False
```

- 与运算 and: 仅当两个操作数的值都等价于 True 时，运算结果为第二个操作数的值，即可能结果为等价于 True 的非布尔型 (bool) 数据；若有至少一个操作数等价于 False，运算结果就是第一个等价于 False 的操作数的值。例如：

```
>>> 3 < 4 and 3 + 6           # 两个操作数都等价于 True
9
>>> 3 + 2 and list()          # 无参的 list()函数返回一个空列表，等价于 False
[]
```

- 或运算 or: 有一个或两个的操作数的值等价于 True，运算结果就为第一个等价于 True 的操作数的值，即可能结果为等价于 True 的非布尔型 (bool) 数据；两个操作数都等价于 False，运算结果为第二个等价于 False 的操作数的值。例如：

```
>>> " or 6 / 7
0.8571428571428571
>>> print(9%3 or None)
None
```

- 逻辑运算中的短路现象：假设表达式 “a 逻辑运算 b”，则在计算 a 后，已经能确定整个表达式结果时，系统不再计算 b 的结果，以提高程序的运行效率。例如：

```
>>> 1 < 0 and tuple()         # 无参的 tuple()函数返回一个空元组，等价于 False
False
```

上述示例中，因为 $1 < 0$ 的结果为 `False`，则 `and` 运算的第二个结果无论是什么，结果都是条件不成立，故系统不再执行第二个操作数对应的操作，即不会生成空元组。

```
>>> 3 + 6 or set()          # 无参的 set()函数返回一个空集合，等价于 False
9
```

上述示例中，因为 $3 + 6$ 的结果为 `9`，等价于 `True`，则 `or` 运算的第二个结果无论是什么，结果都是条件成立，故系统不再执行第二个操作数对应的操作，即不会生成空集合。

4. 1. 3测试运算及函数

1. 成员运算符

Python 中存在序列数据类型和可迭代对象，一个序列数据或可迭代对象中包含若干个数，用户需要测试某数据是否存在于序列或可迭代对象中，就需要使用成员运算符来进行测试。Python 提供了两个成员运算符：`in` 和 `not in`。

`a in b` 在序列或可迭代对象 `b` 中找到值 `a`，就返回 `True`，否则返回 `False`。
`a not in b` 在序列或可迭代对象 `b` 中没有找到值 `a`，就返回 `True`，否则返回 `False`。
示例如下：

```
>>> 8 not in [1, 2, 3, 4, 5]
True
>>> 100 in range(100)        # range 生成范围为[0,100)的整数，包括 0 不包括 100
False
```

2. 身份运算符

Python 的身份运算符用于比较两个操作数是否为同一个对象。Python 提供了两个身份运算符：`is` 和 `not is`。

`a is b` 判断 `a`、`b` 是否引用了同一个对象，是就返回 `True`，否则返回 `False`。
`a not is b` 判断 `a`、`b` 是否引用了不同的对象，是就返回 `True`，否则返回 `False`。

Python 还提供了一个内置函数 `id()` 用来返回变量所引用对象的 `id` 值，该值就是被引用对象的内存地址。参见 2.3 节的介绍。示例如下：

```
>>> x = 20
>>> id(x)
1707017264
>>> y = 20
>>> id(y)
1707017264
>>> x is y
True
>>> a = list()          # a 指向新生成的空列表
>>> id(a)
2271570384968
>>> b = list()          # b 指向新生成的空列表
>>> id(b)
2271570367048
>>> a is not b
True
>>> c = a               # c 执行 a 所指向的空列表，所以 c 和 a 指向同一个对象
```

```
>>> c is not b
True
```

注意：变量被赋予不可变数据和可变数据时，是否引用同一个对象的情况比较复杂，读者可以查阅本书的 3.2.2 和 3.2.3 节的内容。

3. 几个内置的测试函数

- all()函数

all 函数的格式：

```
all(iterable)
```

all()函数用于检测序列或可迭代对象 `iterable` 中的所有元素值是否都不等价于 `False`，如果是返回 `True`，否则返回 `False`。`iterable` 为空也返回 `True`。例如：

```
>>> all(['a', 'b', 'c', 'd'])      # 列表元素都不为空或 0
True
>>> all(['a', 'b', '', 'd'])      # 列表存在一个为空的元素
False
>>> all(('a', 'b', '', 'd'))      # 元组中存在一个为空的元素
False
>>> all({0, 1, 2, 3})             # 集合中存在一个为 0 的元素
False
```

- any()函数

any 函数的格式：

```
any(iterable)
```

any()函数用于检测序列或可迭代对象 `iterable` 中至少有一个元素值等价于 `True`，如果是返回 `True`，否则返回 `False`。`iterable` 为空则返回 `False`。例如：

```
>>> any(['a', 'b', 'c', 'd'])
True
>>> any(['a', 'b', '', 'd'])
True
>>> any({})                       # {}表示空集合
False
```

- isinstance()

格式：

```
isinstance(obj, class_or_tuple)
```

`isinstance()` 函数用来判断一个对象 `obj` 是否是参数 `class_or_tuple` 列举出的类型。例如：

```
>>> isinstance(3, int)
True
>>> isinstance(3, (int, complex, str))
True
```

4.2 条件运算符

当程序控制仅仅只需要根据条件选择不同计算结果时，可以使用条件运算符。Python 的条件运算符的格式为：

```
表达式 1 if 条件表达式 else 表达式 2
```

执行时先计算条件表达式，当条件计算结果为 `True`，返回表达式 1 的计算结果，当条件计算结果为 `False`，则返回表达式 2 的计算结果。

【例 4-1】输入三个数，输出其中最大的数。

程序代码如下：

```
#1. a, b, c = eval(input('a,b,c='))
#2. max = a if a > b else b
#3. max = max if max > c else c
#4. print('Max=', max)
```

测试一的结果：

```
a,b,c=24,56,89✓
Max= 89
```

测试二的结果：

```
a,b,c=50.3,32.9,21.3✓
Max= 50.3
```

测试三的结果：

```
a,b,c=20,68.3,52✓
Max= 68.3
```

4.3 选择结构的实现

Python 只提供了一种选择结构语句：if 语句。if 语句可分为单分支、双分支和多分支三种格式，还可以进行 if 语句的嵌套。

4.3.1 单分支选择结构

单分支结构的 if 语句格式为：

```
if 条件表达式:
    语句块
```

当满足条件只需执行一条语句时，也可以写成如下格式：

```
if 条件表达式: 单语句
```

计算条件表达式时，若结果为 True 或等价于 True，则执行语句块或单语句；若结果为 False 或等价于 False，则执行单分支结构后的后续语句。

注意：

- 条件表达式多数为比较表达式或逻辑表达式，但也可以是其他计算结果的表达式。
- 条件表达式后必须加冒号。
- 相对于 if 所在行，语句块中的所有语句行都应向右缩进对齐，并保持一致的缩进方式。

【例 4-2】编写程序，从键盘输入整数，判定它是否为大于 100 的数。

```
#1. print('The program gets a number, and shows if it is larger than 100.')
#2. print('The number:', end='')
#3. x = int(input('x='))
#4. if x>100:
#5.     print('The number',x,'is larger than 100.')
```

某次的测试结果：

```
The program gets a number, and shows if it is larger than 100.
The number: x=200✓
The number 200 is larger than 100.
```

【例 4-3】输出 3 个整数中的最大数。

```
#1. a, b, c = eval(input('Please input a,b,c:'))
#2. max = a
#3. if b > max:
#4.     max = b
#5. if c > max:
#6.     max = c
#7. print('max=', max)
```

测试一的结果:

```
Please input a,b,c:1,2,3✓
max = 3
```

测试二的结果:

```
Please input a,b,c:2,3,1✓
max = 3
```

测试三的结果:

```
Please input a,b,c:3,1,2✓
max = 3
```

【例 4-4】输入两个整数，从小到大排序输出

```
#1. a, b = eval(input('Please input a,b:'))
#2. if a > b:
#3.     a, b = b, a
#4. print(a, b)
```

测试一的结果:

```
Please input a,b:10,20✓
10 20
```

测试二的结果:

```
Please input a,b:20,10✓
10 20
```

4. 3. 2双分支选择结构

双分支结构的 if 语句格式为:

```
if 条件表达式:
    语句块 1
else:
    语句块 2
```

计算条件表达式时，若结果为 True 或等价于 True，则执行语句块 1；若结果为 False 或等价于 False，则执行语句块 2。

需注意的事项与单分支 if 一致，此外还需注意：

- else 后必须加冒号。
- 语句块 1 和语句块 2 的缩进方式要保持一致。

【例 4-5】使用 if-else 语句改写例 4-4，输入两个整数，从小到大排序输出。

```
#1. a, b = eval(input('Please input a,b:'))
#2. if a < b:
#3.     print(a, b)
#4. else:
```



```
#5.      print(b, a)
```

测试一的结果:

```
Please input a,b:10,20✓
```

```
10 20
```

测试二的结果:

```
Please input a,b:20,10✓
```

```
10 20
```

【例 4-6】使用 if-else 语句改写例 4-3，求三个整数的最大值。

```
#1.  a, b, c = eval(input('Please input a,b,c:'))
```

```
#2.  if a > b:
```

```
#3.      max = a
```

```
#4.  else:
```

```
#5.      max = b
```

```
#6.  if max < c:
```

```
#7.      print('max=', c)
```

```
#8.  else:
```

```
#9.      print('max=', max)
```

测试一的结果:

```
Please input a,b,c:10,20,30✓
```

```
max = 30
```

测试二的结果:

```
Please input a,b,c:20,30,10✓
```

```
max = 30
```

测试三的结果:

```
Please input a,b,c:30,10,20✓
```

```
max = 30
```

4. 3. 3多分支选择结构

多分支结构的 if 语句格式为:

```
if 条件表达式 1:
```

```
    语句块 1
```

```
elif 条件表达式 2:
```

```
    语句块 2
```

```
.....
```

```
else:
```

```
    语句块 n
```

首先计算条件表达式 1，若条件表达式 1 成立，则执行语句块 1，并结束多分支结构；若条件表达式 1 不成立，则计算条件表达式 2，若条件表达式 2 成立，则执行语句块 2，并结束多分支结构；……若所有条件表达式都不成立，则执行 else 后的语句块 n，结束多分支结构。

注意事项与双分支 if 一致，另外还需注意：

- 每个 elif 后的表达式必须加冒号。
- else 子句应书写在最后，并可省略。当省略 else 时，若所有条件表达式都不成立，则不执行任何语句。

- 语句块 1、语句块 2...语句块 n 的缩进方式都应保持一致。

【例 4-7】某大型超市为了促销，采用购物打折优惠方法，每位顾客一次购物：

- ① 在 500 元以上者，按九五折优惠；
- ② 在 1000 元以上者，按九折优惠；
- ③ 在 1500 元以上者，按八五折优惠；
- ④ 在 2000 元以上者，按八折优惠。

编写程序，计算所购商品优惠后的价格。

```
#1. m = eval(input('总金额 = '))
#2. if m < 500:
#3.     d = 1
#4. elif m < 1000:
#5.     d = 0.95
#6. elif m < 1500:
#7.     d = 0.9
#8. elif m < 2000:
#9.     d = 0.85
#10. else:
#11.     d = 0.8
#12. amount = m * d
#13. print('优惠价 =', amount)
```

测试一的结果：

总金额 = 300✓

优惠价 = 300

测试二的结果：

总金额 = 800✓

优惠价 = 760.0

测试三的结果：

总金额 = 1200✓

优惠价 = 1080.0

测试四的结果：

总金额 = 1800✓

优惠价 = 1530.0

测试五的结果：

总金额 = 3000✓

优惠价 = 2400.0

4.3.4 选择结构的嵌套

控制结构的嵌套是指，一个控制结构的内部有另一个控制结构。此处的选择嵌套特指一个选择语句的内部包含另一个选择结构。以下是选择结构嵌套的一种情况：

```
#1. if 条件表达式 1:
#2.     语句块 1
#3.     if 条件表达式 2_1:
#4.         语句块 2_1
#5.     elif 条件表达式 2_2:
```

内层 if 语句

```

#6.         语句块 2_2
#7.         .....
#8.         else:
#9.         语句块 2_n
#10.        语句块 2
#11. elif 条件表达式 2:
#12.        语句块 3
#13. ....
#14. else:
#15.        语句块 n

```

上述的选择结构嵌套中，当条件表达式 1 成立时，执行语句块 1、内层 if 语句和语句块 2，若不成立则继续执行外层 if 语句的其他部分（#11 行及其后的语句）。

在选择嵌套中，外层 if 语句和内层 if 语句可以是单分支、双分支或多分支结构。任何一个语句块中都可以再包含更内层的 if 语句。因 if 语句本身的格式较多，内外层 if 结构的组合可能性也很多，故此不一一列举。

在选择结构嵌套中，因 Python 编程格式要求用缩进来表示同层结构，故前示代码中的 #1、#11、#13 和 #14 行的缩进对齐是一致的，其中 #13 的省略号代表可能包含的多个 elif，也需要和这些行对齐。#2 到 #10 行相对外层 if 向右缩进一层。内层 if 语句从 #3 到 #9 行中，语句块 1、内层 if 语句和语句块 2 是并列的同一层的语句。

Python 的缩进既代表了代码间的层次关系，也增强了程序的可读性。

【例 4-8】使用 if 嵌套改写例 4-3，求三个整数的最大值。

```

#1. a, b, c = eval(input('Please input a,b,c:'))
#2. if a > b and a > c:
#3.     print('max=', a)
#4. else:
#5.     if b > c:
#6.         print('max=', b)
#7.     else:
#8.         print('max=', c)

```

测试一的结果：

```

Please input a,b,c:1,2,3↵
max = 3

```

测试二的结果：

```

Please input a,b,c:1,3,2↵
max = 3

```

测试三的结果：

```

Please input a,b,c:3,2,1↵
max = 3

```

4.4 选择结构程序举例

【例 4-9】编写程序，输入一个百分制的成绩，要求根据不同分数输出成绩等级 A、B、C、D、E。90 分以上为 A，80~89 分为 B，70~79 分为 C，60~69 分为 D，60 分以下为 E。

```

#1. s = eval(input('Please input score='))
#2. if s > 100 or s < 0:

```

```

#3.     print('Input Error!')
#4.  else:
#5.     if s >= 90:
#6.         g = 'A'
#7.     elif s >= 80:
#8.         g = 'B'
#9.     elif s >= 70:
#10.        g = 'C'
#11.    elif s >= 60:
#12.        g = 'D'
#13.    else:
#14.        g = 'E'
#15.    print('Grade is', g)

```

测试一的结果:

Please input score=-1✓
Input Error!

测试二的结果:

Please input score=200✓
Input Error!

测试三的结果:

Please input score=40✓
Grade is E

测试四的结果:

Please input score=65✓
Grade is D

测试五的结果:

Please input score=78✓
Grade is C

测试六的结果:

Please input score=83✓
Grade is B

测试七的结果:

Please input score=92✓
Grade is A

【例 4-10】输入三个整数，从小到大排序输出。

```

#1.  a, b, c = eval(input('Please input a,b,c:'))
#2.  if a > b:
#3.      a, b = b, a
#4.  if a > c:
#5.      a, c = c, a
#6.  if b > c:
#7.      b, c = c, b
#8.  print(a, b, c)

```

测试一的结果:

Please input a,b,c:1,3,2✓

1 2 3

测试二的结果:

Please input a,b,c:3,2,1✓

1 2 3

【例 4-11】改写例 3-5，使之能完全正确地转换。从键盘输入一个 3 位整数，将该整数转换为英文表达。例如，输入 392，输出 three hundred and ninety two。

```
#1. eng1 = ['one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight', 'nine']
#2. eng2 = ['ten', 'twenty', 'thirty', 'forty', 'fifty', 'sixty', 'seventy', 'eighty', 'ninety']
#3. eng3 = ['ten', 'eleven', 'twelve', 'thirteen', 'fourteen', 'fifteen',
#4.         'sixteen', 'seventeen', 'eighteen', 'nineteen']
#5. x = input('请输入一个三位数: ')
#6. if len(x) != 3:
#7.     print('Input Error!')
#8. else:
#9.     a, b, c = map(int, x)
#10.    s = '' # 结果字符串初始为空
#11.    # 直接转换百位，百位不会为 0
#12.    s = eng1[a] + 'hundred'
#13.    if b != 0 or c != 0: # 不是整百的数字
#14.        s += 'and '
#15.        if b == 0: # 十位数为 0 的数字
#16.            s += eng1[c]
#17.        elif b == 1: # 十位数为 1 的数字
#18.            s += eng3[c]
#19.        else:
#20.            s += eng2[b]
#21.            if c != 0: # 个位数为非 0 的数字
#22.                s += '-' + eng1[c]
#23.    print(s)
```

测试一的结果:

请输入一个三位数: 392✓

three hundred and ninety-two

测试二的结果:

请输入一个三位数: 512✓

five hundred and twelve

4.5 习题

1. Python 有哪些关系运算符?
2. 求下列表达式的值:
 - (1) 已有 $x=3$ 、 $y=4$ 、 $z=5$ ，求表达式 $(x+y)+z-1$ and $y+z/2$ 的值。
 - (2) 已有 $x=3$ 、 $y=4$ 、 $z=5$ ，求表达式 $\text{not}(x+y)+z-1$ and $y+z/2$ 的值。
 - (3) 已知 $x=1$ 、 $y=2$ ，求表达式 $x<y$ and $y-1$ 的值。
3. 设 x 的值是 21， y 的值是 4， z 的值是 8， c 的值是 'A'， d 的值是 'H'，请写出下列表达式

的值。

- (1) $x + y \geq z$
- (2) $y == x - 2 * z - 1$
- (3) $6 * x != x$
- (4) $c > d$
- (5) $x = y == 4$
- (6) $2 * c > d$

4. 设 x 的值是 11, y 的值是 6, z 的值是 1, c 的值是 'k', d 的值是 'y', 请写出下列表达式的值。

- (1) $x > 9 \text{ and } y != 3$
- (2) $x == 5 \text{ or } y != 3$
- (3) $\text{not}(x > 14)$
- (4) $\text{not}(x > 9 \text{ and } y != 23)$
- (5) $x \leq 1 \text{ and } y == 6 \text{ or } z < 4$
- (6) $'a' \leq c < 'z'$
- (7) $c \geq 'A' \text{ and } c \leq 'Z'$
- (8) $c != d \text{ and } c != '\n'$
- (9) $5 \text{ and } y != 8 \text{ or } 0$
- (10) $y != 8 \text{ and } 5 \text{ or } 0$

5. 继续改写例 4-11, 使之能将 4 位数转换成英文表达。例如, 1543: one thousand five hundred and forty-three, 2043: two thousand and forty-three。
6. 编写程序, 判断输入的正整数是否为平方数。例如, 144 是平方数, 因为 $144=12^2$, 123 则不是平方数。
7. 编写程序, 接收用户通过键盘输入的 1~13 之间的整数, 将其转换成扑克牌张输出, 1 转换为字符 'A', 2~9 转换为对应的字符, 10 转换为 'T', 11 转换为 'J', 12 转换为 'Q', 13 转换为 'K'。要求使用 if 语句实现。
8. 编写程序, 接收用户输入的年份和月份, 输出该月天数。要求使用 if 语句实现。
9. 编写英寸和厘米之间的双向转换程序。
10. 输入三角形的三条边 a, b, c, 编程判断是否能构成三角形。若可以构成三角形, 则求出三角形面积并判断三角形类型(等边三角形、等腰三角形、直角三角形或一般三角形)。

